

深圳大学考试答题纸

(以论文、报告等形式考核专用)

二〇二三 ~ 二〇二四 学年度第 一 学期

课程编号 1500610003 课序号 03 课程名称 计算机图形学 主讲教师 周漾 评分

2021 级信息与计算科学

学 号 2021192010 姓名 王曦 专业年级 (数学与计算机实验班)

教师评语：

题目：风车吹布料

宋体五号，至少八页，可以从下一页开始写。

成绩评分栏：

评分项	俄罗斯方块文档 (占 12 分)	俄罗斯方块代码 (占 24 分)	俄罗斯方块迟交倒扣分 (占 0 分)	虚拟场景建模文档 (占 16 分)	虚拟场景建模代码 (占 38 分)	演示与答辩 (占 10 分)	虚拟场景建模迟交倒扣分 (占 0 分)	大作业总分
得分								
评分人								

目录

目录.....	2
0. 前言.....	4
0.1 灵感来源.....	4
0.2 项目架构.....	4
0.2.1 文件架构.....	4
0.2.2 代码架构.....	4
0.3 报告说明.....	5
1. 天空盒.....	6
1.1 简介.....	6
1.2 实现.....	6
1.3 效果.....	11
2. 透明材质.....	12
2.1 简介.....	12
2.2 实现.....	12
2.3 效果.....	16
3. 阴影映射.....	18
3.1 简介.....	18
3.2 实现.....	19
3.2.1 渲染深度贴图.....	19
3.2.2 渲染阴影.....	21
3.2.3 阴影偏移.....	24
3.2.4 采样过多.....	25
3.2.5 PCF.....	27
3.3 效果.....	28
4. 聚光.....	29
4.1 简介.....	29
4.2 实现.....	29
4.3 效果.....	31
5. 镜面光贴图.....	32
5.1 简介.....	32
5.2 实现.....	32
5.3 效果.....	33
6. 层级建模.....	34
6.1 简介.....	34
6.2 实现.....	34
6.3 效果.....	39
7. 布料仿真.....	41
7.1 简介.....	41
7.1.1 弹簧质点系统.....	41
7.1.2 用弹簧质点系统模拟布料.....	43
7.1.3 显式 Euler 积分法.....	45
7.2 实现.....	46
7.2.1 质点类.....	46
7.2.2 刚体类.....	47
7.2.3 弹簧类.....	52
7.2.4 布料类.....	52

7.2.5 渲染类.....	57
7.3 效果.....	58
8. Blinn-Phong 光照	60
8.1 简介.....	60
8.2 实现.....	61
8.3 效果.....	62
9. Gamma 校正	63
9.1 简介.....	63
9.2 实现.....	63
9.3 效果.....	64
10. 使用说明书.....	65
10.1 摄像机.....	65
10.1.1 模式切换.....	65
10.1.2 自由摄像机模式.....	66
10.1.3 风车模式.....	66
10.1.4 自动环绕摄像机模式.....	66
10.2 手电筒.....	67
10.3 光影.....	67
10.4 风车.....	68
10.5 布料仿真.....	68
11. 总结.....	69
11.1 基本要求.....	69
11.2 Shining Points.....	69
12. 心得体会.....	70
12.1 工期.....	70
12.2 OpenGL 的调试方法	70
12.3 图形学的学习心得.....	70
参考资料.....	72

0. 前言

0.1 灵感来源

看完 **GAMES 101 现代计算机图形学入门** 后觉得看 **GAMES 102 几何建模与处理** 有点无聊，于是看了一下 **GAMES 103 基于物理的计算机动画入门**，觉得 Lab 做出来的场景很真实，所以做个布料仿真。

本次大作业的布料仿真参考了布料仿真领域的重要论文 Baraff and Witkin. 1998. Large Step in Cloth Simulation. SIGGRAPH (见附件)，建模思路同 **GAMES 103** 的 **Lecture 5 Physics-based Cloth Simulation**，采用弹簧模型进行仿真，实现部分采用本节中的显式积分（而非 Lab 中的隐式积分）。

为使得场景更加真实，用层次建模绘制了一个小风车，用于实现风车吹布料的效果，这便是本次大作业的题目 **风车吹布料** 的来源。

0.2 项目架构

0.2.1 文件架构

以 main.cpp 所在路径为参考。

(1) shaders 文件夹：存放用于绘制各种物体的 shader。

- ① box.vshader/fshader：用于绘制钢框箱子。
- ② cloth.vshader/fshader：用于绘制布料。
- ③ colors.vshader/fshader：用于绘制带阴影的物体。
- ④ debug_depth.vshader/fshader：用于可视化深度贴图。
depth.vshader/fshader：用于绘制深度贴图。
- ⑤ grass.vshader/fshader：用于绘制草。
- ⑥ lightCube.vshader/fshader：用于绘制光源立方体。
- ⑦ rigid.vshader/fshader：用于绘制刚体。
- ⑧ skybox.vshader/fshader：用于绘制天空盒。
- ⑨ spring.vshader/fshader：用于绘制弹簧。
- ⑩ window.vshader/fshader：用于绘制窗户。

(2) textures 文件夹：存放各物体的材质。

0.2.2 代码架构

- (1) main.cpp：主程序。
- (2) camera.h：摄像机类。
- (3) Car.h：风车类。
- (4) cloth.h：布料类。
- (5) CMakeLists.txt：用于 CMake 生成项目。
CMakePresets.json：用于 CMake 的配置文件。
- (6) massPoint.h：质点类。
- (7) render.h：渲染类。

- (8) rigid.h : 刚体类。
- (9) shaders.h : shader 类。
- (10) spring.h : 弹簧类。
- (11) stb_image.h : stb_image 库。

0.3 报告说明

报告的 **第 1 章** 到 **第 10 章** 对本次大作业中用到的技术进行详细的说明，**第 11 章** 指出本次大作业中每个部分分别符合何要求，**第 12 章** 总结了 OpenGL 的调试方法和学习图形学的心得体会。

受篇幅限制，课内的实验中涉及到的技术不作过多介绍，本报告将着重介绍课内的实验未涉及的技术。

1. 天空盒

1.1 简介

正方体贴图（Cube Map）是包含 6 个 2D 纹理的纹理，每个 2D 纹理都是一个面。正方体贴图通过一个方向向量进行采样。

设有如下图所示的 $1 * 1 * 1$ 的单位正方体，方向向量原点位于其中心，用一个橙色的方向向量从正方体贴图上采样纹理值如下图所示。

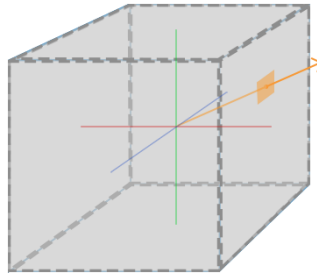


图 1.1.1：正方体贴图的采样示意图

此处的方向向量不关心大小，只关心方向。若正方体中心位于原点，则可用实际向量作为方向向量对正方体贴图进行采样，即可将正方体的顶点位置作为顶点的纹理坐标，得到正方体贴图上的纹理坐标。

天空盒（Skybox）的常见形式如下图所示。

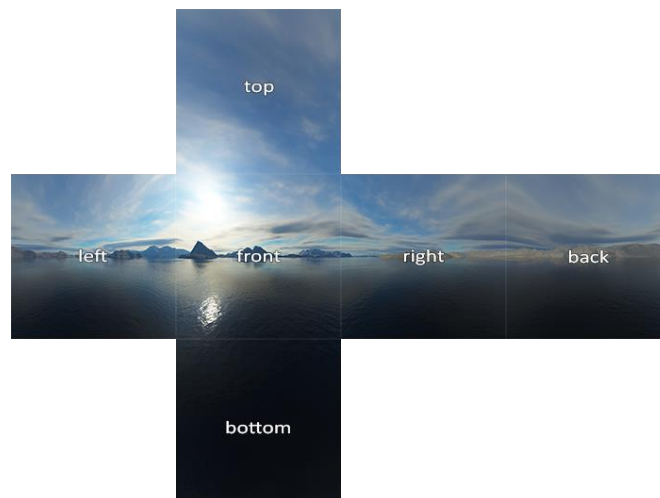


图 1.1.2：天空盒示意图

将六个面折成一个正方体将得到一个有贴图的正方体，可用于模拟一个大场景。

1.2 实现

main.cpp 中实现了函数 loadCubemap() 用于读取正方体贴图，实现如下。

```
// 加载正方体贴图
// 方向: +X (right), -X (left), +Y (top), -Y (bottom), +Z (front), -Z (back)
unsigned int loadCubemap(std::vector<std::string> faces) {
    unsigned int textureID;
    glGenTextures(1, &textureID);
```

```

glBindTexture(GL_TEXTURE_CUBE_MAP, textureID);

int width, height, nrChannels;
for (unsigned int i = 0; i < faces.size(); i++) {
    unsigned char *data = stbi_load(faces[i].c_str(), &width, &height, &nrChannels, 0);
    if (data) {
        glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_X + i, 0, GL_RGB, width,
height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
        stbi_image_free(data);
    }
    else {
        std::cout << "Cubemap texture failed to load at path: " << faces[i] << std::endl;
        stbi_image_free(data);
    }
}

glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_R,
GL_CLAMP_TO_EDGE);
return textureID;
}

```

函数传入一个存有正方体贴图的各面的路径的 `vector`，先生成一个纹理 ID 并将其绑定到 `GL_TEXTURE_CUBE_MAP`，然后用 `stb_image` 库中的 `stbi_load()` 函数加载每个面，再设定每个纹理的环绕和过滤方式。

`skybox.fshader` 中，将一个 `samplerCube` 类型的采样器用于 `texture()` 函数的采样，此处的纹理坐标（即方向向量）为三维向量。

```

in vec3 TexCoords;

uniform samplerCube skybox;

void main() {
    FragColor = texture(skybox, TexCoords);
}

```

本次大作业的代码中将天空盒存放在 `textures` 文件夹中的 `skybox` 文件夹中，`skybox` 文件夹中存放 6 个面的贴图，如下图所示：

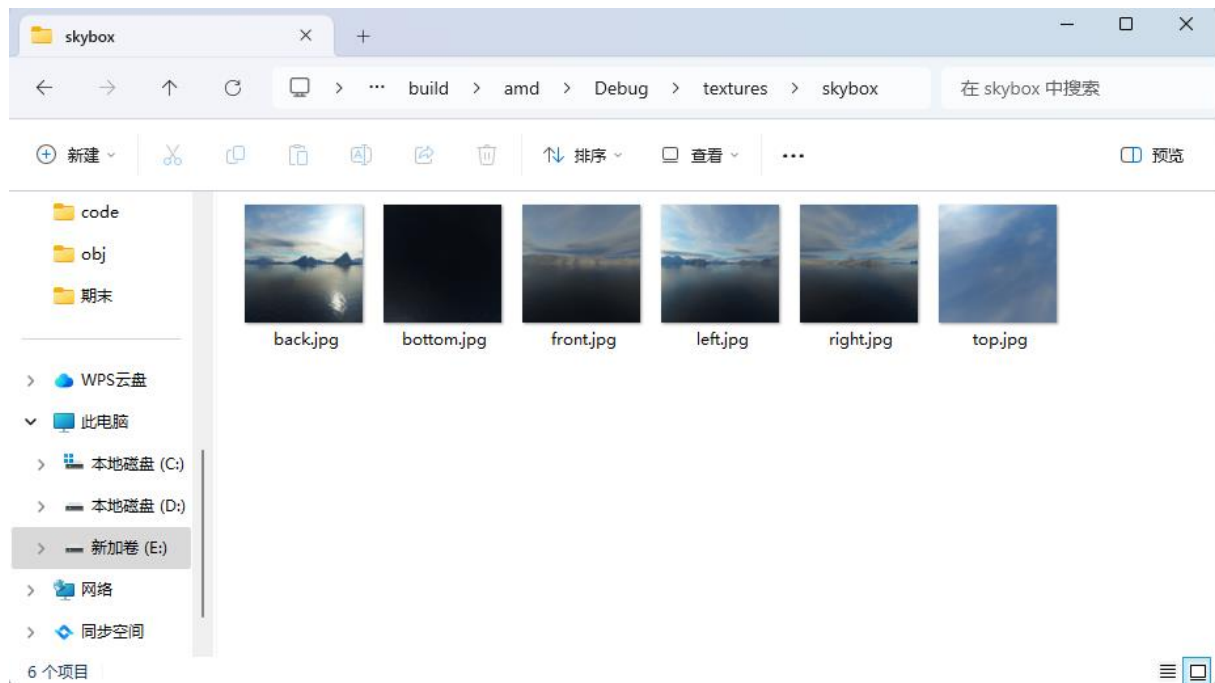


图 1.2.1 : skybox 文件夹

main.cpp 的 main() 函数中将各面贴图的路径存在 vector 中, 并调用 loadCubemap() 函数加载纹理。

```
std::vector<std::string> faces {
    "textures/skybox/right.jpg",
    "textures/skybox/left.jpg",
    "textures/skybox/top.jpg",
    "textures/skybox/bottom.jpg",
    "textures/skybox/front.jpg",
    "textures/skybox/back.jpg"
};

skyboxTexture = loadCubemap(faces); // 天空盒的纹理
```

为防止天空盒被其它物体挡住, 可先禁用深度写入, 渲染时先绘制天空盒再绘制其它物体, 但这样不高效, 因为先渲染天空盒时会对屏幕上的每个像素都运行一遍 fshader, 而实际中并非天空盒的所有部分都可见, 故可采用提前深度测试 (Early Depth Testing) 丢弃天空盒被挡住的部分, 在绘制其它不透明的物体 (含阴影) 后再绘制天空盒, 可节约带宽。

为使得天空盒始终绘制在其它物体之后, 可设定天空盒的深度为 1.0, 这可在 skybox.vshader 中通过透视除法实现, 代码如下。

```
layout (location = 0) in vec3 aPos;

out vec3 TexCoords;

uniform mat4 view;
uniform mat4 projection;

void main() {
    TexCoords = aPos;
    vec4 pos = projection * view * vec4(aPos, 1.0);
    gl_Position = pos.xyww;
}
```


因归一化坐标中只能有一个等于 1.0 的 z 值，即最大深度，则能让天空盒在无可见物体时才渲染。

main.cpp 中绘制完其它不透明的物体（含阴影）后，用如下方式绘制天空盒。

```
// 绘制天空盒时将深度函数改为 <=
glDepthFunc(GL_LEQUAL);

// 设置矩阵
skyboxShader.use();
view = glm::mat4(glm::mat3(camera.GetViewMatrix())); // 移除位移
skyboxShader.setMat4("view", view);
skyboxShader.setMat4("projection", projection);

// 激活并绑定纹理
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, skyboxTexture);

// 绘制正方体
renderCube();

// 恢复深度函数
glDepthFunc(GL_LESS);
```

因绘制正方体是常用操作，故将其封装为 car.h 中的 renderCube() 函数，其实现如下。

```
unsigned int cubeVAO = 0, cubeVBO = 0; // 正方体的 VAO 和 VBO

// 在归一化坐标中渲染一个 2 * 2 * 2 的正方体
void renderCube() {
    if (cubeVAO == 0) { // 初始化
        // 正方体的顶点
        float vertices[] = {
            // 顶点坐标          法向量          纹理坐标
            // 背面
            -1.0f, -1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 0.0f, 0.0f, // 左下角
            1.0f,  1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 1.0f, 1.0f, // 右上角
            1.0f, -1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 1.0f, 0.0f, // 右下角
            1.0f,  1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 1.0f, 1.0f, // 右上角
            -1.0f, -1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 0.0f, 0.0f, // 左下角
            -1.0f,  1.0f, -1.0f,  0.0f,  0.0f, -1.0f, 0.0f, 1.0f, // 左上角
            // 正面
            -1.0f, -1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 0.0f, 0.0f, // 左下角
            1.0f, -1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 1.0f, 0.0f, // 右下角
            1.0f,  1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 1.0f, 1.0f, // 右上角
            1.0f,  1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 1.0f, 1.0f, // 右上角
            -1.0f,  1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 0.0f, 1.0f, // 左上角
            -1.0f, -1.0f,  1.0f,  0.0f,  0.0f,  1.0f, 0.0f, 0.0f, // 左下角
            // 左面
            -1.0f,  1.0f,  1.0f, -1.0f,  0.0f,  0.0f, 1.0f, 0.0f, // 右上角
            -1.0f,  1.0f, -1.0f, -1.0f,  0.0f,  0.0f, 1.0f, 1.0f, // 左上角
```

```

-1.0f, -1.0f, -1.0f, -1.0f,  0.0f,  0.0f, 0.0f, 1.0f, // 左下角
-1.0f, -1.0f, -1.0f, -1.0f,  0.0f,  0.0f, 0.0f, 1.0f, // 左下角
-1.0f, -1.0f,  1.0f, -1.0f,  0.0f,  0.0f, 0.0f, 0.0f, // 右下角
-1.0f,  1.0f,  1.0f, -1.0f,  0.0f,  0.0f, 1.0f, 0.0f, // 右上角
// 右面
1.0f,  1.0f,  1.0f,  1.0f,  0.0f,  0.0f, 1.0f, 0.0f, // 左上角
1.0f, -1.0f, -1.0f,  1.0f,  0.0f,  0.0f, 0.0f, 1.0f, // 右下角
1.0f,  1.0f, -1.0f,  1.0f,  0.0f,  0.0f, 1.0f, 1.0f, // 右上角
1.0f, -1.0f, -1.0f,  1.0f,  0.0f,  0.0f, 0.0f, 1.0f, // 右下角
1.0f,  1.0f,  1.0f,  1.0f,  0.0f,  0.0f, 1.0f, 0.0f, // 左上角
1.0f, -1.0f,  1.0f,  1.0f,  0.0f,  0.0f, 0.0f, 0.0f, // 左下角
// 底面
-1.0f, -1.0f, -1.0f,  0.0f, -1.0f,  0.0f, 0.0f, 1.0f, // 右上角
1.0f, -1.0f, -1.0f,  0.0f, -1.0f,  0.0f, 1.0f, 1.0f, // 左上角
1.0f, -1.0f,  1.0f,  0.0f, -1.0f,  0.0f, 1.0f, 0.0f, // 左下角
1.0f, -1.0f,  1.0f,  0.0f, -1.0f,  0.0f, 1.0f, 0.0f, // 左下角
-1.0f, -1.0f,  1.0f,  0.0f, -1.0f,  0.0f, 0.0f, 0.0f, // 右下角
-1.0f, -1.0f, -1.0f,  0.0f, -1.0f,  0.0f, 0.0f, 1.0f, // 右上角
// 顶面
-1.0f,  1.0f, -1.0f,  0.0f,  1.0f,  0.0f, 0.0f, 1.0f, // 左上角
1.0f,  1.0f,  1.0f,  0.0f,  1.0f,  0.0f, 1.0f, 0.0f, // 右下角
1.0f,  1.0f, -1.0f,  0.0f,  1.0f,  0.0f, 1.0f, 1.0f, // 右上角
1.0f,  1.0f,  1.0f,  0.0f,  1.0f,  0.0f, 1.0f, 0.0f, // 右下角
-1.0f,  1.0f, -1.0f,  0.0f,  1.0f,  0.0f, 0.0f, 1.0f, // 左上角
-1.0f,  1.0f,  1.0f,  0.0f,  1.0f,  0.0f, 0.0f, 0.0f // 左下角
};

glGenBuffers(1, &cubeVBO);
glBindBuffer(GL_ARRAY_BUFFER, cubeVBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glGenVertexArrays(1, &cubeVAO);
glBindVertexArray(cubeVAO);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);

glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
}

// 绘制正方体
glBindVertexArray(cubeVAO);
glDrawArrays(GL_TRIANGLES, 0, 36);

```

```
glBindVertexArray(0);  
}
```

绘制天空盒时，将深度测试的深度函数设置为 `GL_LEQUAL`，这样不会让天空盒被其它物体挡住。绘制完天空盒后，将深度函数恢复为 `GL_LESS`。

设置 view 矩阵时，用

```
view = glm::mat4(glm::mat3(camera.GetViewMatrix())); // 移除位移
```

取摄像机 camera 的 view 矩阵中左上角的 3×3 矩阵，即移除了位移部分，使得无论照相机如何移动，天空盒都不会移动，营造玩家处于很大的场景中的感觉。

1.3 效果

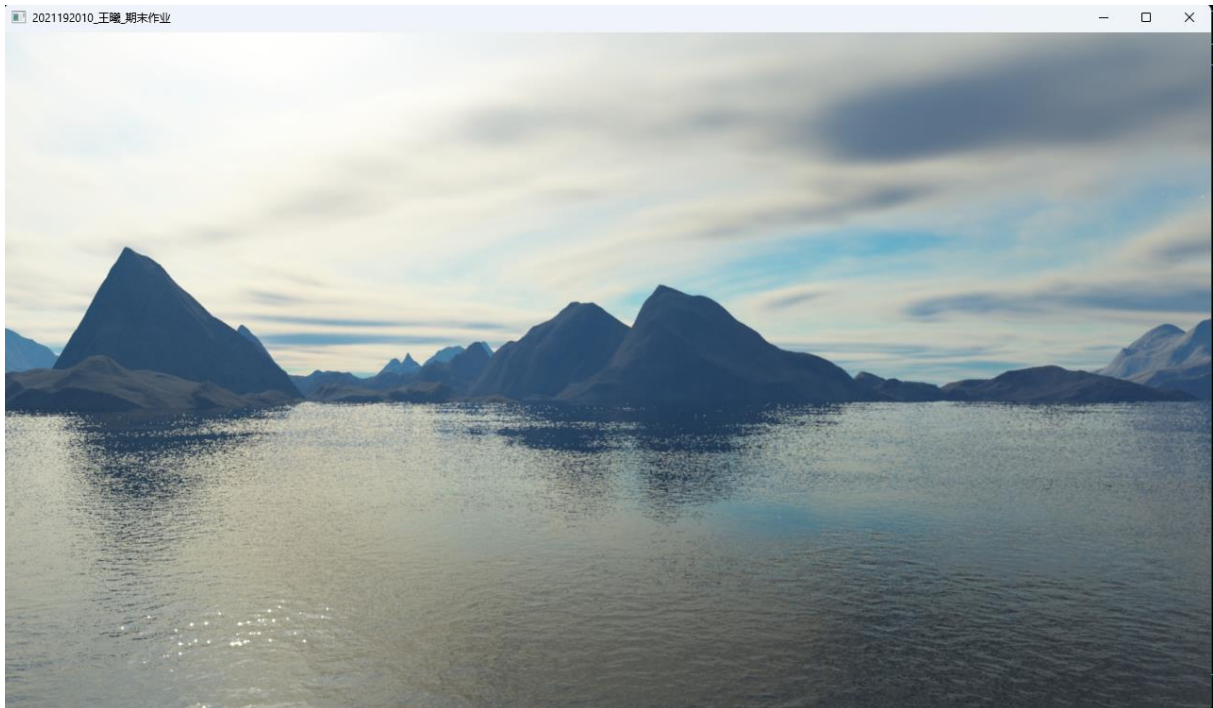


图 1.3.1：天空盒渲染效果

2. 透明材质

2.1 简介

OpenGL 中，**混合**（Blending）是实现物体透明度的技术，其中**透明**指物体或其一部分的颜色不是纯色，而是由物体本身的颜色与其背后的物体的颜色的不同强度的结合。

完全透明的窗户、红色的半透明窗户的示意图如下。



图 2.1.1：全透明、半透明示意图

有些材质除了 R、G、B 分量外还有 alpha 通道，每个像素都包含一个 alpha 值，该值可精确地处理纹理每部分的透明度。

2.2 实现

render.h 中实现了函数 loadTexture() 用于读取 2D 纹理，当纹理图片有 alpha 通道时会自动加载。

采样纹理边缘时，OpenGL 会对边缘的值和纹理下一个重复的值进行插值（因为环绕方式为 GL_REPEAT），这在只使用 RGB 值时没问题。但使用 alpha 值时，纹理的顶部将与底部边缘的纯色值进行插值，产生一个半透明的有色边框，可能导致渲染处的全透明的物体周围有一个半透明的有色边框。为避免该情况，在有 alpha 通道时将环绕模式设为 GL_CLAMP_TO_EDGE 即可。

```
// 加载纹理
unsigned int loadTexture(char const* path) {
    unsigned int textureID;
    glGenTextures(1, &textureID);

    int width, height, nrComponents;
    unsigned char* data = stbi_load(path, &width, &height, &nrComponents, 0);
    if (data) {
        GLenum format;
        if (nrComponents == 1) format = GL_RED;
        else if (nrComponents == 3) format = GL_RGB;
        else if (nrComponents == 4) format = GL_RGBA;
```

```

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format,
GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);

        // 有 alpha 纹理时, 以 GL_CLAMP_TO_EDGE 为环绕方式以避免看到半透明的纹理四
        边形
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, format == GL_RGBA ?
GL_CLAMP_TO_EDGE : GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, format == GL_RGBA ?
GL_CLAMP_TO_EDGE : GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

        stbi_image_free(data);
    }
    else {
        std::cout << "Texture failed to load at path: " << path << std::endl;
        stbi_image_free(data);
    }
    return textureID;
}

```

为渲染全透明的材质, 如草, 可在 grass.fshader 中丢弃 $\alpha < 0.1$ 的片段, 代码如下。

```

out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture1;

void main() {
    vec4 texColor = texture(texture1, TexCoords);
    if (texColor.a < 0.1) discard; // 丢弃不透明度 < 0.1 的片段

    FragColor = texture(texture1, TexCoords);
}

```

main.cpp 中渲染 4 棵全透明的草的代码如下。

```

// -----= 草 =-----

// 草正方形的顶点
float grassVertices[] = {
    // 顶点坐标          // 纹理坐标
    0.0f, 0.5f, 0.0f, 0.0f, 0.0f,
    0.0f, -0.5f, 0.0f, 0.0f, 1.0f,
    1.0f, -0.5f, 0.0f, 1.0f, 1.0f,

    0.0f, 0.5f, 0.0f, 0.0f, 0.0f,

```

```

        1.0f, -0.5f, 0.0f, 1.0f, 1.0f,
        1.0f, 0.5f, 0.0f, 1.0f, 0.0f
    };// 草的位置

    std::vector<glm::vec3> grassPosition {
        glm::vec3(5.0f, 0.0f, 5.0f),
        glm::vec3(6.0f, 0.0f, 6.0f),
        glm::vec3(7.0f, 0.0f, 7.0f),
        glm::vec3(8.0f, 0.0f, 8.0f),
    };

    ... // 设置草的 VAO 、 VBO ， 加载纹理等

    // 设置矩阵
    grassShader.use();
    view = camera.GetViewMatrix();
    projection = glm::perspective(glm::radians(camera.Zoom), (float)SCR_WIDTH /
(float)SCR_HEIGHT, 0.1f, 100.0f);
    grassShader.setMat4("view", view);
    grassShader.setMat4("projection", projection);

    // 激活并绑定草的纹理
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, grassTexture);

    // 绑定草的 VAO
    glBindVertexArray(grassVAO);

    // 绘制草
    for (unsigned int i = 0; i < grassPosition.size(); i++) {
        model = glm::mat4(1.0f);
        model = glm::translate(model, grassPosition[i]);
        model = glm::rotate(model, glm::radians(i * 45.0f), glm::vec3(0.0f, 1.0f, 0.0f));
        grassShader.setMat4("model", model);

        // 绘制四边形的 6 个顶点
        glDrawArrays(GL_TRIANGLES, 0, 6);
    }

    // 解绑 VAO
    glBindVertexArray(0);

```

渲染半透明图像不能采取丢弃片段的方法，而要用混合。OpenGL 中的混合方程如下：

$$\vec{C}_{\text{result}} = \vec{C}_{\text{source}} \cdot F_{\text{source}} + \vec{C}_{\text{destination}} \cdot F_{\text{destination}}$$

其中：

符号	说明
----	----

$\vec{C}_{\text{source}}$	源颜色向量，是来自纹理的颜色向量。
F_{source}	源因子值，指定了 alpha 值对源颜色的影响。
$\vec{C}_{\text{destination}}$	目标颜色向量，是存储在颜色缓冲中的颜色向量。
$F_{\text{destination}}$	目标因子值，指定了 alpha 值对目标颜色的影响。

fshader 运行完成且所有测试都通过后，混合方程将应用于片段颜色输出与当前颜色缓冲中的值。

源因子和目标颜色由 OpenGL 自动设定，源因子值可自己指定。为渲染 图 2.1.1 的右图所示的有色窗户，应取源颜色的 alpha 值作源因子，取 (1 - alpha) 作目标因子，开启混合函数后，用 GL_ONE_MINUS_SRC_ALPHA, GL_ONE, GL_ZERO 作混合函数即可，代码如下。

```
glEnable(GL_BLEND); // 启用混合
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA); // 设置半透明的混合函数
```

用于渲染窗户的 window.vshader 和 window.fshader 与渲染普通物体的 vshader 和 fshader 类似。

同时开启深度测试和混合时，深度缓冲不会检查片段是否透明，这导致透过前一个窗户看后一个窗户时，后一个窗户在前一个窗户的透明部分的片段会被丢弃，即前一个窗户挡住后一个窗户。为解决该问题，需按顺序绘制半透明的物体，即先渲染离当前摄像机位置远的半透明物体，再渲染离当前摄像机近的半透明物体。

```
// ===== 窗户 =====

// 窗户正方形的顶点
float windowVertices[] = {
    // 顶点坐标          // 纹理坐标
    0.0f, 0.5f, 0.0f, 0.0f, 0.0f,
    0.0f, -0.5f, 0.0f, 0.0f, 1.0f,
    1.0f, -0.5f, 0.0f, 1.0f, 1.0f,

    0.0f, 0.5f, 0.0f, 0.0f, 0.0f,
    1.0f, -0.5f, 0.0f, 1.0f, 1.0f,
    1.0f, 0.5f, 0.0f, 1.0f, 0.0f
};

// 窗户的位置
std::vector<glm::vec3> windowPosition {
    glm::vec3(11.0f, 1.0f, 11.0f),
    glm::vec3(6.0f, 1.0f, 12.0f),
};

// ===== 绘制草 =====

// 设置矩阵
grassShader.use();
view = camera.GetViewMatrix();
projection = glm::perspective(glm::radians(camera.Zoom), (float)SCR_WIDTH /
```

```

(float)SCR_HEIGHT, 0.1f, 100.0f);
    grassShader.setMat4("view", view);
    grassShader.setMat4("projection", projection);

    // 激活并绑定草的纹理
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, grassTexture);

    // 绑定草的 VAO
    glBindVertexArray(grassVAO);

    // 绘制草
    for (unsigned int i = 0; i < grassPosition.size(); i++) {
        model = glm::mat4(1.0f);
        model = glm::translate(model, grassPosition[i]);
        model = glm::rotate(model, glm::radians(i * 45.0f), glm::vec3(0.0f, 1.0f, 0.0f));
        grassShader.setMat4("model", model);

        // 绘制四边形的 6 个顶点
        glDrawArrays(GL_TRIANGLES, 0, 6);
    }

    // 解绑 VAO
    glBindVertexArray(0);

```

有天空盒、不透明物体、全透明物体、半透明物体的场景的渲染顺序如下：

- ① 不透明物体、全透明物体。
- ② 天空盒。
- ③ 按顺序的半透明物体。

其中天空盒在半透明物体之前绘制是为了透过半透明物体也能看见天空盒。

2.3 效果



图 2.3.1 : 全透明的草

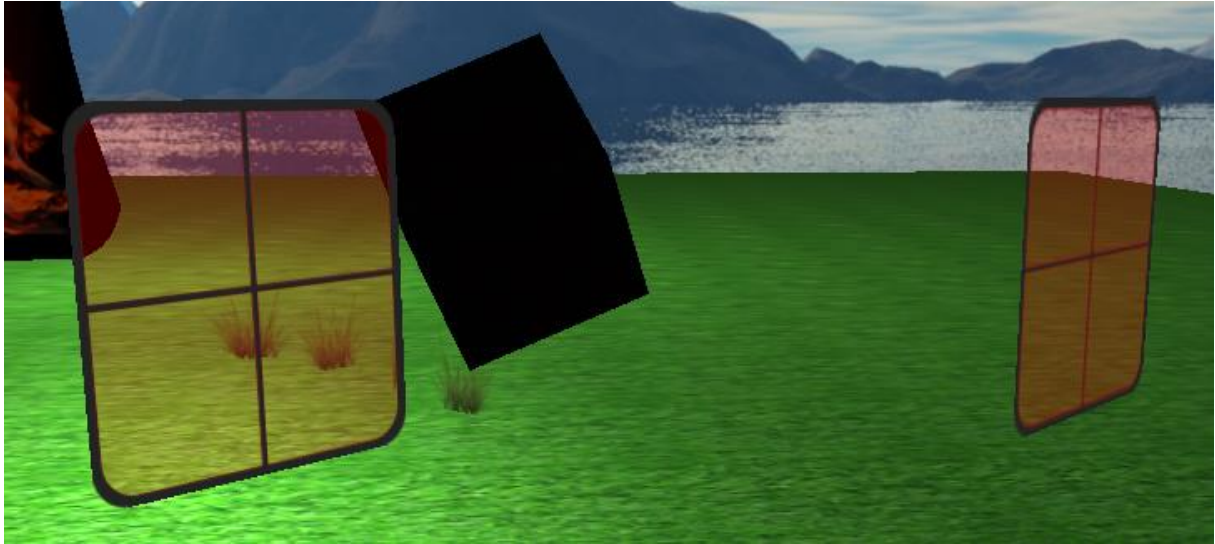


图 2.3.2 : 透过半透明的窗户看物体 (含全透明的物体、天空盒)

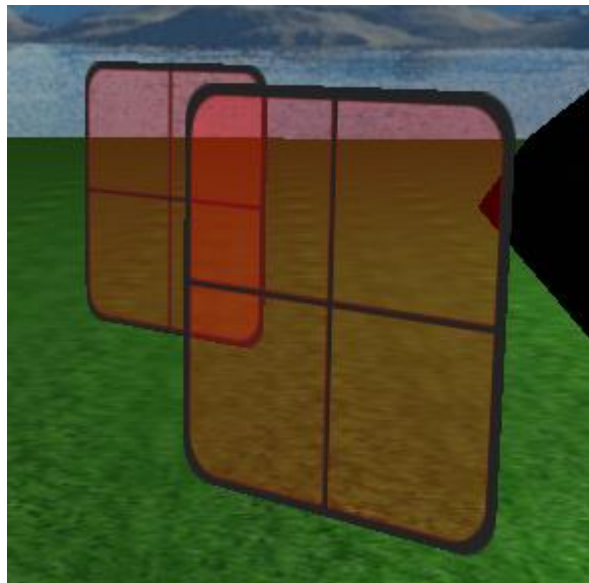


图 2.3.3 : 透过半透明的窗户看半透明的窗户

3. 阴影映射

3.1 简介

阴影映射（Shadow Mapping）的思路：以光的视角渲染，能看到的東西被点亮，看不到的在阴影中。

设场景有地板、光源、3 个箱子，如下图所示。

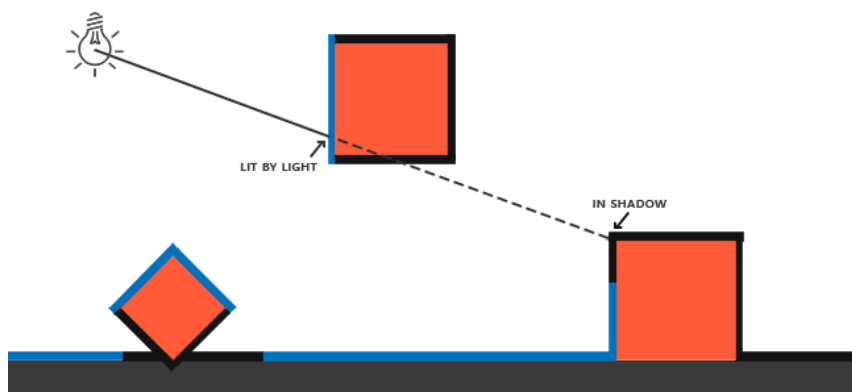


图 3.1.1：场景示意图

上图中蓝线表示光源可看见的片段；黑线表示光源不可看见的片段，它们应渲染为带阴影的。

从光源发出成千上万的射线进行遍历是消耗性能的，基本无法应用于实时渲染。实时渲染中的阴影映射的实现借助深度缓冲。

从光源的透视图渲染场景，将深度值的结果存储到纹理中，这样可采样光源的透视图所见的最近深度，深度值显示从光源的透视图下见到的第一个片段。存储在纹理中的这些深度值称为**阴影贴图（Shadow Map）**或**深度贴图（Depth Map）**，它可通过用光源的视图和投影矩阵渲染场景得到。

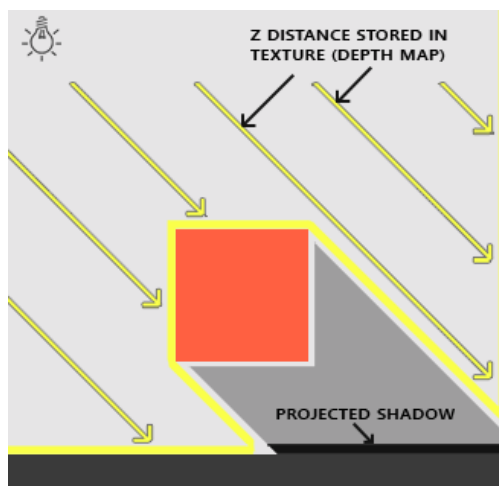


图 3.1.2：阴影贴图示意图

上图的左图展示了一个定向光（所有光线平行）在正方体表面投影的阴影。通过深度贴图上的深度值可找到最近点，用于确定该片段是否在阴影中。

阴影映射由两部分组成：

- ① 用光源视图渲染深度贴图。
- ② 渲染场景，用深度贴图判断片段是否在阴影中。

3.2 实现

3.2.1 渲染深度贴图

main.cpp 的 main() 函数中渲染深度贴图的代码如下。

```
// -----= 深度贴图 = -----

const unsigned int SHADOW_WIDTH = 1024, SHADOW_HEIGHT = 1024; // 深度贴图的分辨率

// 设置深度的帧缓冲对象
unsigned int depthMapFBO;
glGenFramebuffers(1, &depthMapFBO);

// 设置深度贴图
unsigned int depthMap;
glGenTextures(1, &depthMap);
glBindTexture(GL_TEXTURE_2D, depthMap);

// 加载深度贴图
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT, SHADOW_WIDTH,
SHADOW_HEIGHT, 0, GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameterfv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, borderColor);

// 将深度贴图绑定到帧缓冲
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D,
depthMap, 0);

// 将读写缓冲设为 GL_NONE，表示不渲染任何颜色数据
glDrawBuffer(GL_NONE);
glReadBuffer(GL_NONE);
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

上述代码先为深度贴图生成一个帧缓冲对象，然后创建一个 2D 纹理供帧缓冲的深度缓冲使用。

纹理的高、宽设定为 1024，它是深度贴图的分辨率。

因只关注深度贴图的深度值，则将纹理格式设定为 GL_DEPTH_COMPONENT。虽不关注深度贴图的颜色值时无需使用颜色缓冲，但不包含颜色缓冲的帧缓冲对象不完整，故需显式地告诉 OpenGL 不渲染颜色数据。上述代码中的

```
// 将读写缓冲设为 GL_NONE，表示不渲染任何颜色数据
glDrawBuffer(GL_NONE);
glReadBuffer(GL_NONE);
```

将读写缓冲设置为 GL_NONE 以实现该效果。

将深度值渲染到纹理的帧缓冲后，即可开始渲染深度贴图。

因渲染深度贴图时使用定向光，则需对光源使用正交投影矩阵。下面代码中计算处出的矩阵 lightSpaceMatrix 能将世界坐标中的坐标变换到光源视角的空间中的坐标。

```
// -----= 渲染深度贴图 =-----  
glm::mat4 lightProjection, lightView;  
glm::mat4 lightSpaceMatrix;  
float near_plane = 1.0f, far_plane = 7.5f;  
lightProjection = glm::ortho(-10.0f, 10.0f, -10.0f, 10.0f, near_plane, far_plane);  
lightView = glm::lookAt(lightPos, glm::vec3(0.0f), glm::vec3(0.0, 1.0, 0.0));  
lightSpaceMatrix = lightProjection * lightView;
```

在此基础上，给 shader 提供光空间的投影和视图矩阵即可渲染场景。因需关心深度贴图的深度值，为提升性能，将一个简单的 vshader 和一个空的 fshader 用于渲染深度贴图。

depth.vshader 将顶点用 lightSpaceMatrix 变换到光空间中，其代码如下。

```
layout (location = 0) in vec3 aPos;  
  
uniform mat4 lightSpaceMatrix;  
uniform mat4 model;  
  
void main() {  
    // 将顶点坐标变换到光空间  
    gl_Position = lightSpaceMatrix * model * vec4(aPos, 1.0);  
}
```

depth.fshader 可为空，也可显式地设置深度缓冲。但底层默认设置深度缓冲，故注释掉更高效。其代码如下。

```
void main() {  
    // 因无颜色缓冲，则片段无需处理，用空的 fshader 即可  
    // 也可取消注释，显式设置深度，但底层默认设置深度缓冲，故注释更高效  
    // gl_FragDepth = gl_FragCoord.z;  
}
```

用如下代码渲染场景的深度贴图，其中 renderScene() 为渲染场景的函数，向其传入参数 depthShader 表示用深度贴图的 shader 用于渲染场景。

```
renderScene(depthShader);
```

为可视化深度贴图，可将其渲染到四边形上，完成该工作的 debug_depth.fshader 的代码如下。

```
out vec4 FragColor;  
  
in vec2 TexCoords;  
  
uniform sampler2D depthMap;  
uniform float near_plane;  
uniform float far_plane;  
  
// 使用透视投影时需该函数  
float LinearizeDepth(float depth) {  
    float z = depth * 2.0 - 1.0; // 转回归一化坐标  
    return (2.0 * near_plane * far_plane) / (far_plane + near_plane - z * (far_plane - near_plane));  
}
```

```

}

void main() {
    float depthValue = texture(depthMap, TexCoords).r;

    // 透视投影
    // FragColor = vec4(vec3(LinearizeDepth(depthValue) / far_plane), 1.0);

    // 正交投影
    FragColor = vec4(vec3(depthValue), 1.0);
}

```

渲染深度贴图时，使用正交投影矩阵和使用透视投影矩阵稍有不同。正交投影矩阵不将场景用透视图变形，但透视投影矩阵会将顶点按透视关系变形，它们产生的阴影区域的不同如下图所示。

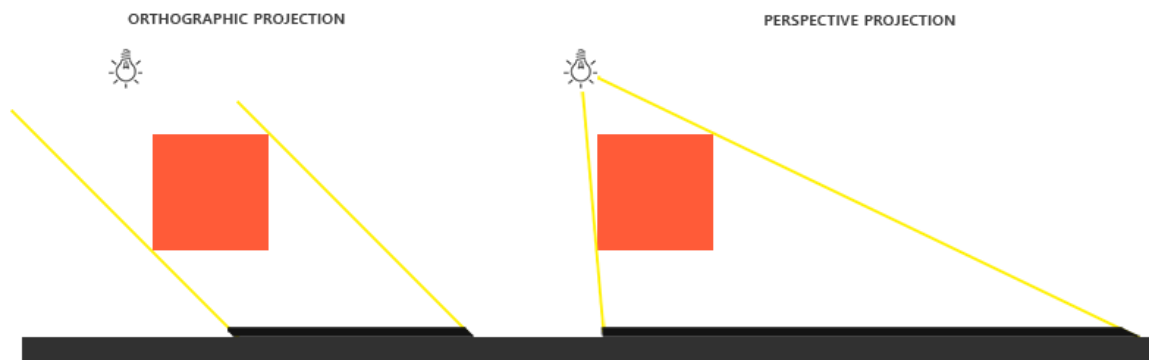


图 3.2.1.1：正交投影与透视投影产生的阴影对比

由于该差异，透视投影常用于点光源、聚光灯，正交投影常用于定向光。

3.2.2 渲染阴影

在用于渲染带阴影的物体的 `colors.vshaders` 中做光空间变换，除了向 `colors.fshader` 传递顶点在世界坐标中的位置 `vs_out.FragPos` 外，还传递顶点在光空间中的位置 `vs_out.FragPosLightSpace`。

```

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoords;

out vec2 TexCoords;

out VS_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec4 FragPosLightSpace;
} vs_out;

uniform mat4 projection;
uniform mat4 view;
uniform mat4 model;
uniform mat4 lightSpaceMatrix;

```

```

void main() {
    vs_out.FragPos = vec3(model * vec4(aPos, 1.0));
    vs_out.Normal = transpose(inverse(mat3(model))) * aNormal;
    vs_out.TexCoords = aTexCoords;
    vs_out.FragPosLightSpace = lightSpaceMatrix * vec4(vs_out.FragPos, 1.0);
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}

```

生成渲染贴图后，可用其渲染阴影，该工作在用于渲染带阴影的物体的 `colors.fshader` 中进行，其代码如下。

```

out vec4 FragColor;

in VS_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec4 FragPosLightSpace;
} fs_in;

uniform sampler2D diffuseTexture;
uniform sampler2D shadowMap;

uniform vec3 lightPos;
uniform vec3 viewPos;

uniform bool blinn;

float ShadowCalculation(vec4 fragPosLightSpace, vec3 normal, vec3 lightDir) {
    // 透视除法
    vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;

    // 映射到 [0,1] 范围内
    projCoords = projCoords * 0.5 + 0.5;

    // 求透视光方向的最近深度
    float closestDepth = texture(shadowMap, projCoords.xy).r;

    // 求透视光方向的当前片段的深度
    float currentDepth = projCoords.z;

    // 检查当前片段是否在阴影中
    float shadow = currentDepth > closestDepth ? 1.0 : 0.0;
    return shadow;
}

void main() {
    vec3 color = texture(diffuseTexture, fs_in.TexCoords).rgb;
    vec3 normal = normalize(fs_in.Normal);
}

```

```

vec3 lightColor = vec3(1.0f);

// 环境光
vec3 ambient = 0.3 * lightColor;

// 漫反射
vec3 lightDir = normalize(lightPos - fs_in.FragPos);
float diff = max(dot(lightDir, normal), 0.0);
vec3 diffuse = diff * lightColor;

// 镜面反射
vec3 viewDir = normalize(viewPos - fs_in.FragPos);
vec3 reflectDir = reflect(-lightDir, normal);
float spec = 0.0;
if (blinn) { // Blinn-Phong 模型
    vec3 halfwayDir = normalize(lightDir + viewDir);
    spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0);
}
else {
    vec3 reflectDir = reflect(-lightDir, normal);
    spec = pow(max(dot(viewDir, reflectDir), 0.0), 8.0);
}
vec3 specular = spec * lightColor;

// 阴影
float shadow = ShadowCalculation(fs_in.FragPosLightSpace, normal, lightDir);
vec3 lighting = (ambient + (1.0 - shadow) * (diffuse + specular)) * color;

FragColor = vec4(lighting, 1.0);
}

```

colors.fshader 中采用 Blinn-Phong 光照（将在 **8. Blinn-Phong 光照** 介绍）计算光照后，计算 shadow 值，当片段在阴影中时为 1.0，在阴影外时为 0.0，再让光照的 diffuse 和 specular 分量乘 (1 - shadow)，此处不给分量 ambient 乘因为阴影不是全黑的（因为散射）。

因裁切空间的 FragPosLightSpace 不通过 gl_Position 传到 fshader 中，故需手动做透视除法。colors.fshader 中的函数 ShadowCalculation() 函数先做透视除法，再将原本 [-1, 1] 的范围映射到 [0, 1]。事实上，使用正交投影时，顶点的 w 不变，故做透视除法无意义，但使用透视投影时，做透视除法时必须的。

得到投影坐标 projCoords 后，可在深度贴图采样得到 [0, 1] 的结果，得到光的视野下的最近深度 closestDepth 和片段的深度 currentDepth，比较它们的大小即可判断片段是否在阴影中，返回对应的 shadow 值即可。

用上述方法渲染得到的阴影如下图所示。

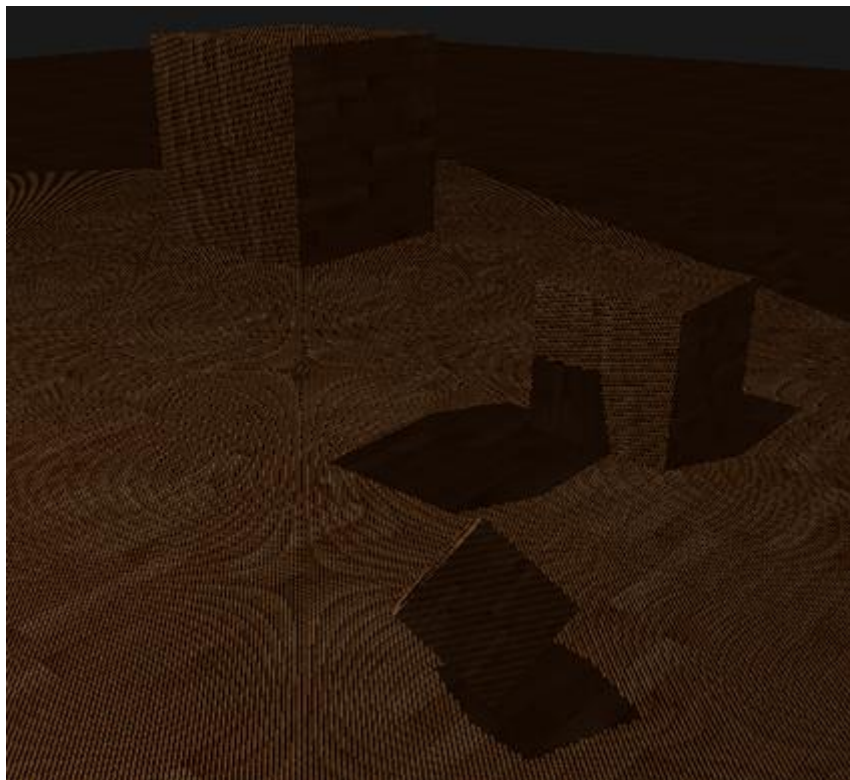


图 3.2.2.1 ：渲染阴影第一阶段

3.2.3 阴影偏移

图 3.2.2.1 中的地面上有交替的黑线，该现象称为**阴影失真**（Shadow Acne）。

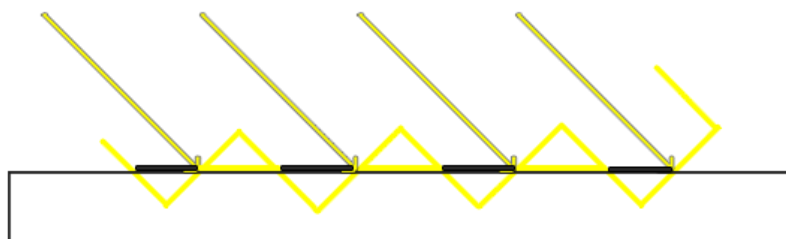


图 3.2.3.1 ：阴影失真成因图

因阴影贴图受分辨率限制，在距光源较远的情况下，多个片段可能从深度贴图的同一值采样。上图中的每个斜坡表示深度贴图的一个单独的纹理像素。观察到，多个片段会从同一深度值采样。

当光源以一个角度朝向表面时会产生阴影失真，因为此时深度贴图也是在一个角度下渲染的，则多个片段从同一斜坡的深度纹理像素中采样有些在地面上，有些在地面下，进而有些片段被认为在阴影中，有些被认为不在阴影中，故产生了 图 3.2.2.1 中的条纹样式。

可用**阴影偏移**（Shadow Bias）解决该问题，它对表面深度（或深度贴图）加一个偏移量，这样片段不会被误认为在地面之下，示意图如下图所示。

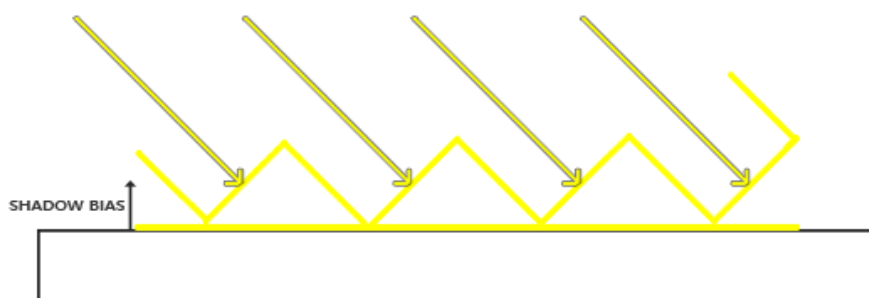


图 3.2.3.2 : 阴影偏移示意图

在 colors.fshader 中取偏移量 bias 进行阴影偏移, 根据表面朝向光线的角度计算得到 [0.005, 0.05] 范围内的 bias , 代码如下。

```
// 检查当前片段是否在阴影中
float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005); // 阴影偏移

float shadow = currentDepth - bias > closestDepth ? 1.0 : 0.0;
```

使用阴影偏移后的效果如下图所示。



图 3.2.3.3 : 使用阴影偏移后渲染的阴影

3.2.4 采样过多

注意到 图 3.2.2.1 中远处的地面被错误地添加阴影, 这是因为阴影映射认为光的视锥不可见的区域都在阴影中, 此时若超出光的视锥的投影坐标大于 0.1 , 则采样的深度纹理超出 [0, 1] 的范围。上述被误认为在阴影中的区域实际上是深度贴图的大小, 深度贴图影响地板是因为其环绕方式为 GL_REPEAT 。

为解决该问题, 可让所有超出阴影贴图的坐标的深度为 1.0 , 这样超出的部分被认为不在阴影中, 再将深度贴图的环绕方式设置为 GL_CLAMP_TO_BORDER , 并设置超出范围后的返回 1.0 的深度值。

上述过程的代码如下。

```
// 将深度贴图的环境方式设为 GL_CLAMP_TO_BORDER s.t. 采样深度贴图 [0, 1] 范围以外的区域时,
// 纹理函数总返回 1.0 的深度值和 0.0 的阴影值, 避免采样过多
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_BORDER);
GLfloat borderColor[] = { 1.0, 1.0, 1.0, 1.0 };
glTexParameterfv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, borderColor);
```

此时渲染的效果如下图所示。

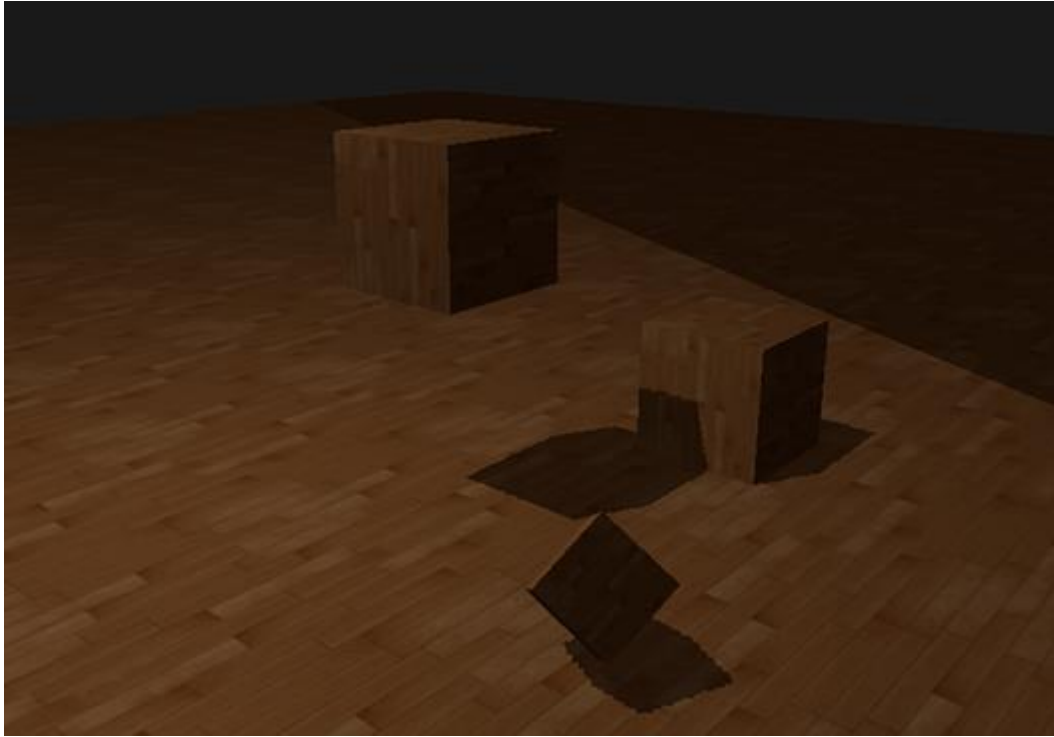


图 3.2.4.1：渲染阴影的第二阶段

注意到上图中地面仍有一部分区域被误认为在阴影中，该区域中的坐标超出光的正交视锥的远平面，此时其投影坐标的 $z > 0.1$ ，环绕方式 `GL_CLAMP_TO_BORDER` 不起作用。为解决该问题，可在 `colors.fshader` 中将 $z > 0.1$ 的投影向量对应的 `shadow` 值设为 `0.0`，代码如下。

```
// 若点比光的远平面还远，则环绕方式 GL_CLAMP_TO_BORDER 无效，此时需手动设置 shadow
if (projCoords.z > 1.0) shadow = 0.0;
```

此时的渲染效果如下图所示。



图 3.2.4.2：渲染阴影的第三阶段

3.2.5 PCF

注意到 图 3.2.4.2 中的阴影有较多锯齿，这是因为深度贴图的分辨率一定，多个片段对应同一纹理像素时会从深度贴图的同一深度采样，这几个片段将得到同一阴影，产生锯齿边。

可用 **PCF**（Percentage-closer Filtering）技术产生柔和的阴影，减少锯齿和硬边的出现。它从深度贴图中多次采样，每次采样的纹理坐标稍有不同，结合多次采样的结果取平均，即可得到柔和的阴影。

在 colors.fshader 中加入 PCF，代码如下。

```
float shadow = 0.0;
if (pcf) {
    // PCF：从纹理像素四周对深度贴图采样，对结果取平均
    vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
    for(int x = -1; x <= 1; ++x) {
        for(int y = -1; y <= 1; ++y) {
            float pcfDepth = texture(shadowMap, projCoords.xy + vec2(x, y) * texelSize).r;
            shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
        }
    }
    shadow /= 9.0;
}
else shadow = currentDepth - bias > closestDepth ? 1.0 : 0.0;
```

加入 PCF 后的渲染效果如下图所示。



图 3.2.5.1：使用 PCF 渲染的阴影

3.3 效果

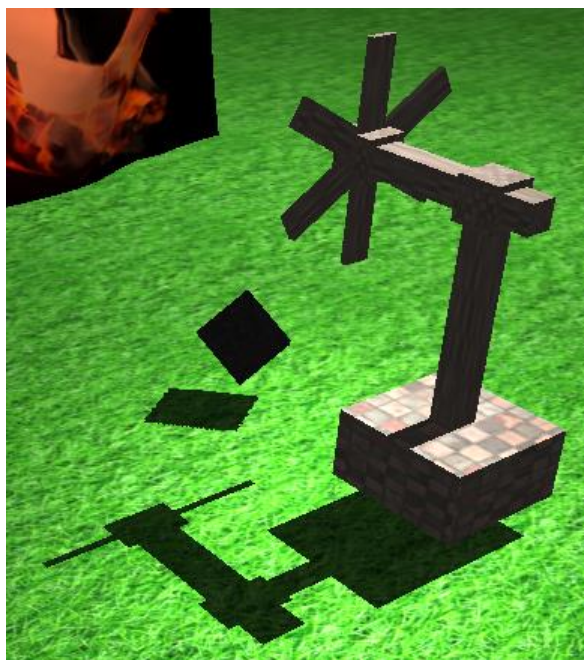


图 3.3.1 ：在地面上和物体本身上的阴影（关闭 PCF ）



图 3.3.2 ：在其它物体上的阴影 （启用 PCF ）

4. 聚光

4.1 简介

聚光（Spotlight）是位于环境中某位置的光源，它只朝着特定方向而非所有方向照射光线，在聚光方向的特定半径内的物体才会被照亮，其它物体保持黑暗，如路灯、手电筒。

OpenGL 中的聚光用一个位置坐标、一个方向向量和一个**切光角**（Cutoff Angle）表示，其中切光角指定了聚光的半径，即圆锥的地面半径。对每个片段，判断它是否在聚光的切光方向间，即是否在圆锥内，若是，则照亮对应的片段。其示意图如下图所示。

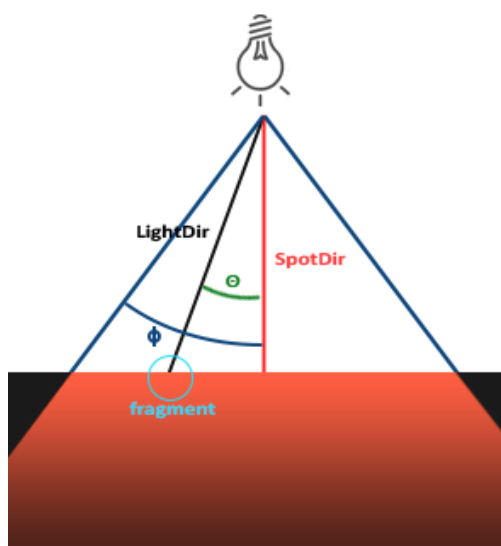


图 4.1.1 ：聚光示意图

其中：

符号	含义
LightDir	从片段指向光源的向量。
SpotDir	聚光所指方向。
ϕ	指定聚光半径的切光角，落在该角度外的物体不会被该聚光照亮。
θ	向量 LightDir 与向量 SpotDir 的夹角。在聚光内部，有 $\theta < \phi$ 。

手电筒（Flashlight）是位于观察者位置的聚光，通常瞄准玩家视角的正前方。

4.2 实现

绘制钢框木箱的 box.fshader 中用结构体 Light 描述聚光，其代码如下。

```
struct Light {  
    vec3 position; // 光源位置  
    vec3 direction; // 光源方向  
  
    // 切光角  
    float cutOff; // 内圆锥  
    float outerCutOff; // 外圆锥
```

```

vec3 ambient; // 环境光
vec3 diffuse; // 漫反射
vec3 specular; // 镜面反射

float constant; // 常数项
float linear; // 一次项
float quadratic; // 二次项
};

```

其中将切光角分为了内圆锥和外圆锥，片段位于内圆锥中时，用如下方法计算 Phong 模型中的各分量。

```

// 漫反射
vec3 norm = normalize(Normal);
vec3 lightDir = normalize(light.position - FragPos);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = light.diffuse * diff * texture(material.diffuse, TexCoords).rgb;

// 镜面反射
vec3 viewDir = normalize(viewPos - FragPos);
vec3 reflectDir = reflect(-lightDir, norm);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);
vec3 specular = light.specular * spec * texture(material.specular, TexCoords).rgb;

// 聚光
...

// 衰减系数
float distance = length(light.position - FragPos);
float attenuation = 1.0 / (light.constant + light.linear * distance + light.quadratic * (distance * distance));
ambient *= attenuation;
diffuse *= attenuation;
specular *= attenuation;

```

注意到在聚光的圆锥的边缘两侧的片段一亮一暗，会让聚光产生一圈硬边，如下图所示。

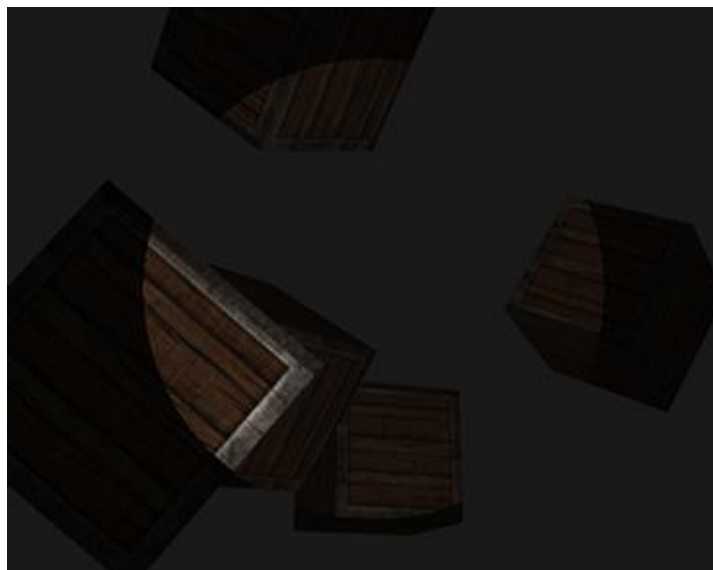


图 4.2.1：聚光有一圈硬边

为手电筒效果显得更加真实，可加入外圆锥，位于内外圆锥间的片段用公式

$$I = \frac{\theta - \gamma}{\varepsilon}$$

计算光强 I ，其中 ε 为内外圆锥切光角 Φ 与外圆锥 γ 间的余弦差。
此时的渲染效果如下图所示。

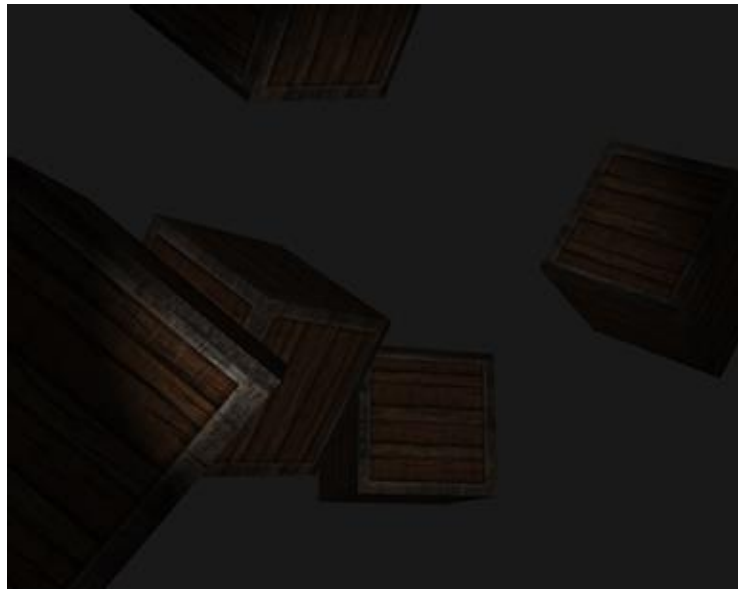
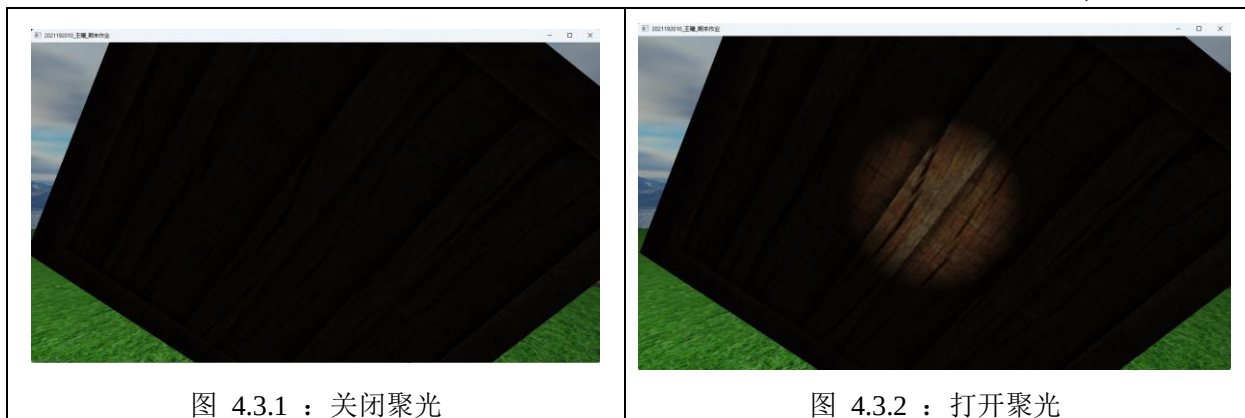


图 4.2.2：聚光边缘软化

4.3 效果

调节各参数的值以在场景中达到较真实的效果。本次大作业中取 $\Phi = 6.5$ ， $\gamma = 12.5$ 。



5. 镜面光贴图

5.1 简介

现实生活中，光照到钢框箱子时，中间的木制部分反光不明显，边缘的钢框有明显的反光。为在 OpenGL 中实现该效果，可在木箱的漫反射光贴图（Diffuse Map）的基础上添加一层钢框的镜面光贴图（Specular Map），如下图所示。



图 5.1.1：木箱的漫反射光贴图

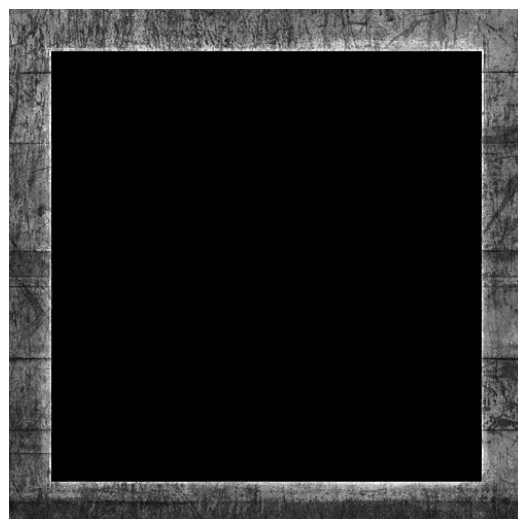


图 5.1.2：镜面光贴图

5.2 实现

在用于绘制钢框木箱的 box.fshader 中用结构体 Material 刻画钢框木箱的材质，其中忽略环境光是因为环境光几乎都等于漫反射；定义一个 sampler2D 类型的镜面光分量 specular。

```
struct Material {  
    sampler2D diffuse; // 只保留漫反射，因为环境光几乎都等于漫反射  
    sampler2D specular; // 镜面反射  
    float shininess; // 反光度  
};
```

在 main.cpp 的 main() 函数中用如下代码加载钢框的漫反射贴图和镜面光贴图。

```
boxDiffuseTexture = loadTexture("textures/container2.png"); // 钢框箱子的漫反射纹理  
boxSpecularTexture = loadTexture("textures/container2_specular.png"); // 钢框箱子的镜面反射纹理
```

在 main.cpp 的 main() 函数中向 shader 传递参数、激活并绑定漫反射纹理和镜面反射纹理，绘制钢框木箱，代码如下。

```
// -----= 绘制钢框箱子 =-----  
boxShader.use();  
// 设置矩阵  
projection = glm::perspective(glm::radians(camera.Zoom), (float)SCR_WIDTH /  
(float)SCR_HEIGHT, 0.1f, 100.0f);  
boxShader.setMat4("projection", projection);
```



```

view = camera.GetViewMatrix();
boxShader.setMat4("view", view);
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(8.0f, 1.2f, 8.0f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(1.0f, 0.3f, 0.5f));
model = glm::scale(model, glm::vec3(0.75f));
boxShader.setMat4("model", model);
// 设置材质
boxShader.setInt("material.diffuse", 0);
boxShader.setInt("material.specular", 1);
boxShader.setFloat("material.shininess", 32.0f);
// 设置聚光
boxShader.setInt("torch", torch);
boxShader.setFloat("light.cutOff", glm::cos(glm::radians(6.5f)));
boxShader.setFloat("light.outerCutOff", glm::cos(glm::radians(12.5f)));
// 设置光源
boxShader.setVec3("light.position", camera.Position);
boxShader.setVec3("light.direction", camera.Front);
boxShader.setVec3("viewPos", camera.Position);
boxShader.setVec3("light.ambient", 0.1f, 0.1f, 0.1f);
boxShader.setVec3("light.diffuse", 0.75f, 0.75f, 0.75f);
boxShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
boxShader.setFloat("light.constant", 1.0f);
boxShader.setFloat("light.linear", 0.09f);
boxShader.setFloat("light.quadratic", 0.032f);
// 激活并绑定漫反射纹理
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, boxDiffuseTexture);
// 激活并绑定镜面反射纹理
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, boxSpecularTexture);
// 绘制正方体
renderCube();

```

5.3 效果

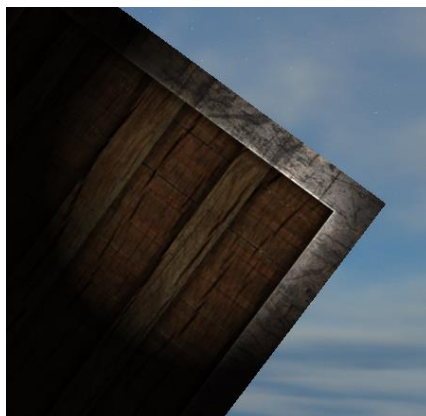


图 5.3.1 : 聚光照射的钢框部分有反光效果

6. 层级建模

6.1 简介

有些物体由多个子模块拼接而成，如汽车由轮胎、车身等拼接而成，这些模型的构建涉及到多个物体的组合，此时需考虑物体间的相对位置、物体运动时各子模块的运动等问题，这需要**层级建模**的技术。

用树形或有向无环图型的结构表示层级建模。下图展示了机器人的层级建模。

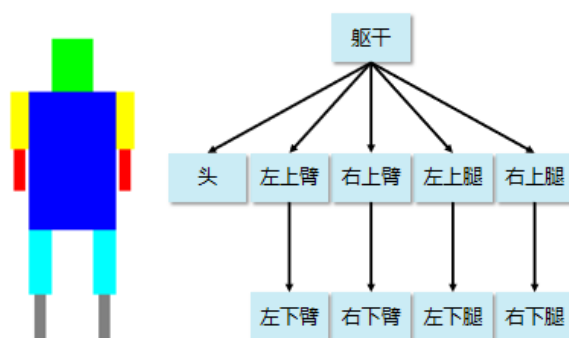


图 6.1.1：机器人的层级建模示意图

绘制物体时，可设定一个位置坐标和朝向向量，根据它们确定每个子模块所在的位置。

物体运动时，将作用于指定子模块的变换矩阵下传给其所有后继，该过程可用 DFS 或栈实现。若层数较少，也可暴力列出每个子模块的变换矩阵。

6.2 实现

本次大作业用层次建模绘制了如下的风车，其层次建模示意图如下图所示。

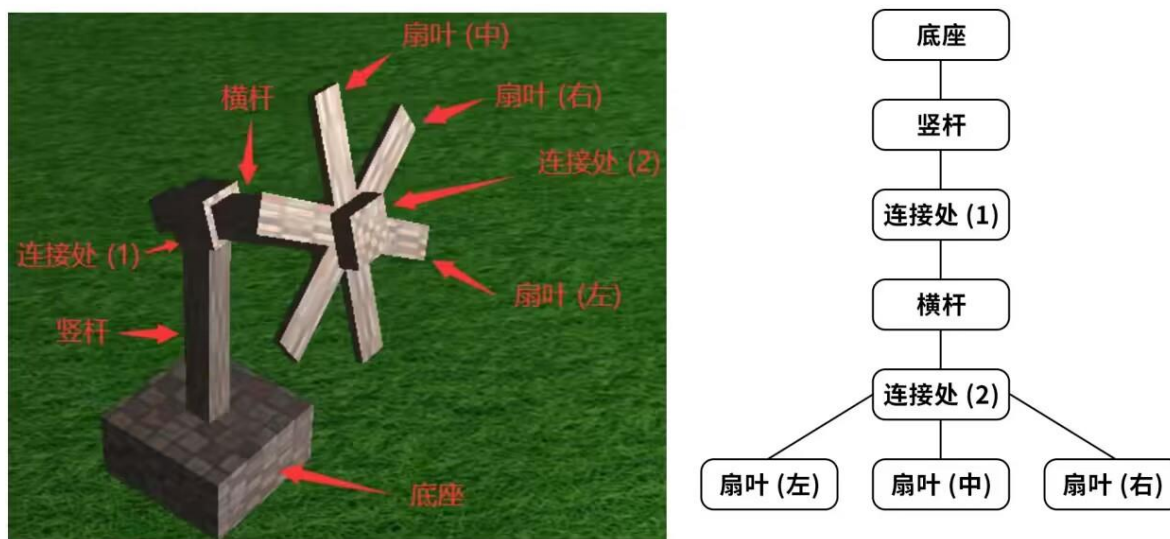


图 6.2.1：风车的层次建模示意图

为使得在摄像机在风车模式下可用键盘的 W、A、S、D 键控制风车的前后左右移动，用鼠标移动控制风车的视角（连接处 (2) 和各扇叶的朝向），风车类（car.h）的成员变量类似于摄像机类（camera.h），只去掉了摄像机的视野 ZOOM。

风车的位置移动、视角变换同摄像机，采用 Euler 角（偏航角 yaw 和俯仰角 pitch 实现）更

新 front 向量、right 向量、up 向量实现，代码如下。

```
// 用 Euler 角更新向量 Front 、 Right 、 Up
void updateCarVectors() {
    glm::vec3 _front;
    _front.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
    _front.y = sin(glm::radians(pitch));
    _front.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
    front = glm::normalize(_front);

    // 单位化向量的原因：向上或向下看会导致向量长度趋于 0，导致移动变慢
    right = glm::normalize(glm::cross(front, worldUp));
    up = glm::normalize(glm::cross(right, front));
}

// 处理键盘输入
void processKeyboard(CarMovement direction, float deltaTime) {
    float velocity = movementSpeed * deltaTime;
    if (direction == CAR_FORWARD) // 前移
        position += front * velocity;
    else if (direction == CAR_BACKWARD) // 后移
        position -= front * velocity;
    else if (direction == CAR_LEFT) // 左移
        position -= right * velocity;
    else if (direction == CAR_RIGHT) // 右移
        position += right * velocity;

    if (position.y < -0.24) position.y = -0.24;
}

// 处理鼠标输入
void processMouseMovement(float xoffset, float yoffset, GLboolean constrainPitch = true) {
    xoffset *= mouseSensitivity;
    yoffset *= mouseSensitivity;

    yaw += xoffset;
    pitch += yoffset;

    // 将俯仰角控制在 [-89.0, 89.0] 范围内，避免屏幕翻转
    if (constrainPitch) {
        if (pitch > 89.0f) pitch = 89.0f;
        if (pitch < -89.0f) pitch = -89.0f;
    }

    // 更新向量 Front 、 Right 、 Up
    updateCarVectors();
}
```

风车扇叶的旋转有两种实现方式：

- ① 对风车的连接处 (2) 或扇叶施加一个旋转矩阵。

② 改变绘制扇叶时的初始角度 `theta` 。

本次大作业采用上述的实现 ②，`bool` 变量 `blowing` 为 `true` 时风车旋转，用一个 `deltaTheta` 更新 `theta` 即可，`deltaTheta` 的大小反映风速大小，后续将用于风力控制。风车的层级建模和扇叶的旋转的代码如下。

```
// 绘制车
void draw(const Shader& shader) {
    // 车身
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::translate(model, position);
    // model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.5f, 0.25f, 0.5f));
    shader.setMat4("model", model);
    renderCube();

    // 竖杆
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(position.x, position.y + 1.0f, position.z));
    // model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.75f, 0.1f));
    shader.setMat4("model", model);
    renderCube();

    // 连接处
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
    // model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
    model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
    // model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));
    shader.setMat4("model", model);
    renderCube();

    // 横杆
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
    // model = glm::translate(model, camera.Front * 0.25f);
    model = glm::translate(model, front * 0.25f);
    // model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
    model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
    // model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.6f, 0.1f, 0.1f));
    shader.setMat4("model", model);
    renderCube();
}
```

```

// 连接处
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
// model = glm::translate(model, camera.Front * 1.0f);
model = glm::translate(model, front * 1.0f);
// model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
// model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));
shader.setMat4("model", model);
renderCube();

if (blowing) {
    theta += deltaTheta;
    if (theta >= 180.0f) theta = 0.0f;
}

// 扇叶(上)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
// model = glm::translate(model, camera.Front * 1.0f);
model = glm::translate(model, front * 1.0f);
// model = glm::rotate(model, glm::radians(0.0f), camera.Front);
model = glm::rotate(model, glm::radians(theta - 60.0f), front);
// model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
// model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.025f, 0.75f, 0.10f));
shader.setMat4("model", model);
renderCube();

// 扇叶(右)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
// model = glm::translate(model, glm::normalize(camera.Front) * 1.0f);
model = glm::translate(model, glm::normalize(front) * 1.0f);
// model = glm::rotate(model, glm::radians(60.0f), camera.Front);
model = glm::rotate(model, glm::radians(theta), front);
// model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
// model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.025f, 0.75f, 0.10f));
shader.setMat4("model", model);
renderCube();

```

```

// 扇叶(左)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(position.x, position.y + 1.75f, position.z));
// model = glm::translate(model, glm::normalize(camera.Front) * 1.0f);
model = glm::translate(model, glm::normalize(front) * 1.0f);
// model = glm::rotate(model, glm::radians(-60.0f), camera.Front);
model = glm::rotate(model, glm::radians(theta + 60.0f), front);
// model = glm::rotate(model, glm::radians(camera.Pitch), glm::normalize(camera.Right));
model = glm::rotate(model, glm::radians(pitch), glm::normalize(right));
// model = glm::rotate(model, -glm::radians(camera.Yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -glm::radians(yaw), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.025f, 0.75f, 0.10f));
shader.setMat4("model", model);
renderCube();
}

```

风车的吹风效果的实现：根据风车的朝向和风力的大小，给布料施加一个对应方向的力。

风车的朝向判定：考察过风车的连接处 (2) 的位置坐标的点 P、以风车的 front 向量为方向向量的直线 l。对布料的每个三角面片，求其所在平面与直线 l 的交点 Q，判断 Q 是否在该三角面片中，并判断向量 PQ 与风车的 front 向量是否同向。判断向量同向时，注意特判除以 0 的情况。

若上述两条件都满足，则认为风车朝着布料吹。这种假设的合理性：布料被风车吹起的部分会离开直线 l 穿过的范围，离开的部分将不再受风车的风力，该部分受重力下落，下落后可能再次回到直线 l 穿过的范围，再次被吹起，如此循环往复，实现风车吹起布料的效果。代码如下。

```

// 控制风车是否开启
bool blowingKeyPressed = false;

// 键盘输入的回调函数
void processInput(GLFWwindow* window) {
    ...

    // -----= 仿真 =-----
    // C：控制风车是否开启
    if (glfwGetKey(window, GLFW_KEY_C) == GLFW_PRESS && !blowingKeyPressed) {
        car.blowing ^= 1;
        blowingKeyPressed = true;

        if (car.blowing) std::cout << "[Windmill] On" << std::endl;
        else std::cout << "[Windmill] Off" << std::endl;
    }
    if (glfwGetKey(window, GLFW_KEY_C) == GLFW_RELEASE) {
        blowingKeyPressed = false;
    }

    ...
}

// 风

```

```

glm::vec3 wind;
glm::vec3 windStartPosition; // 风的起点
glm::vec3 windDirection; // 风向
bool windBlowing = false; // 是否刮风
float windForceScale = 10.0f; // 风力的大小

int main() {
    ...

    // 渲染循环
    while (!glfwWindowShouldClose(window)) {
        ...

        if (running) {
            ...

            // 风车
            if (car.blowing) {
                windStartPosition = car.position;
                windDirection = glm::normalize(car.front);
                windForceScale = car.deltaTheta;
                wind = windDirection * windForceScale;
                cloth.applyForce(wind);
            }
        }
    }
}

```

布料仿真将在 7. 布料仿真 介绍。

6.3 效果

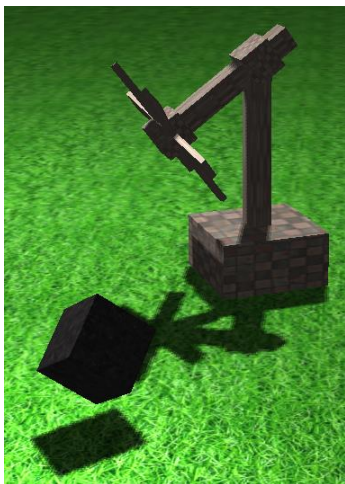


图 6.3.1 : 风车的位置移动与视角变换 (前)



图 6.3.2 : 风车的位置移动与视角变换 (后)

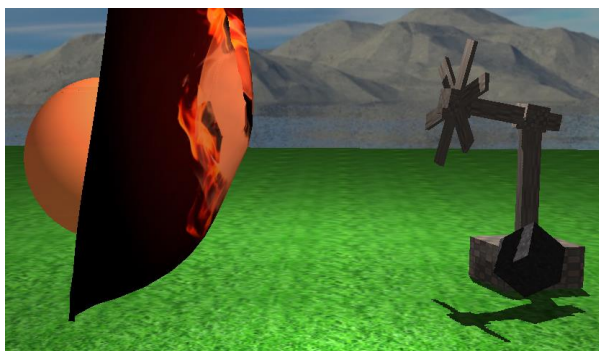


图 6.3.3 : 关闭风车

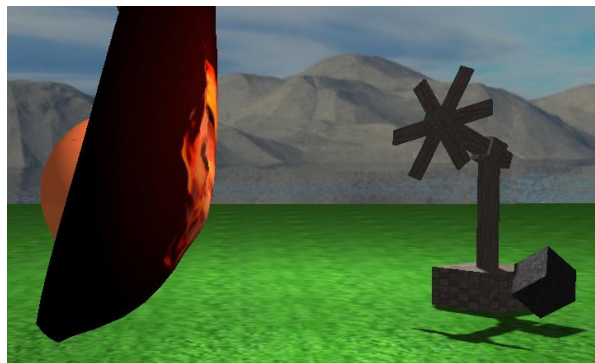


图 6.3.4 : 开启风车

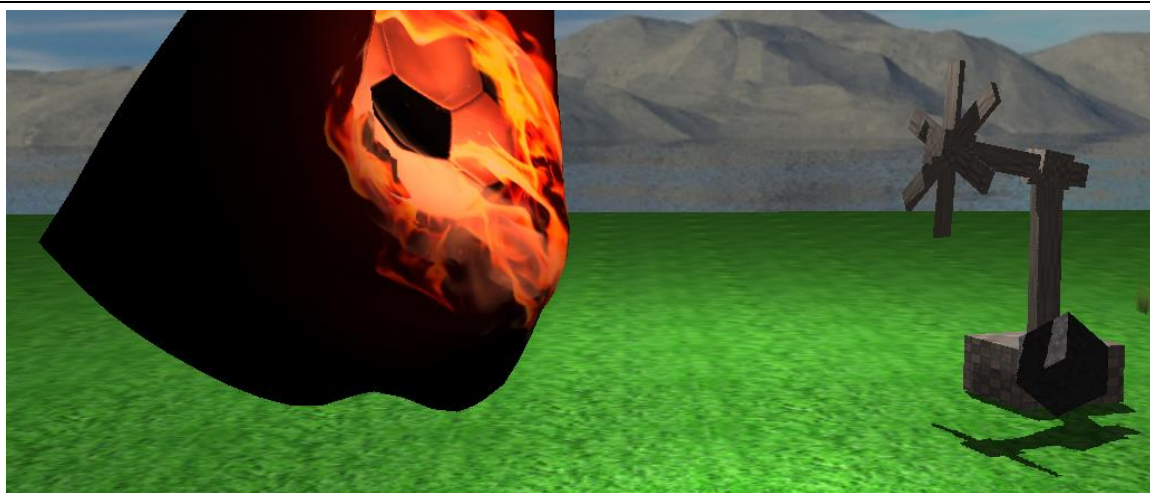


图 6.3.5 : 增大风力（一部分布料在球上滑行）

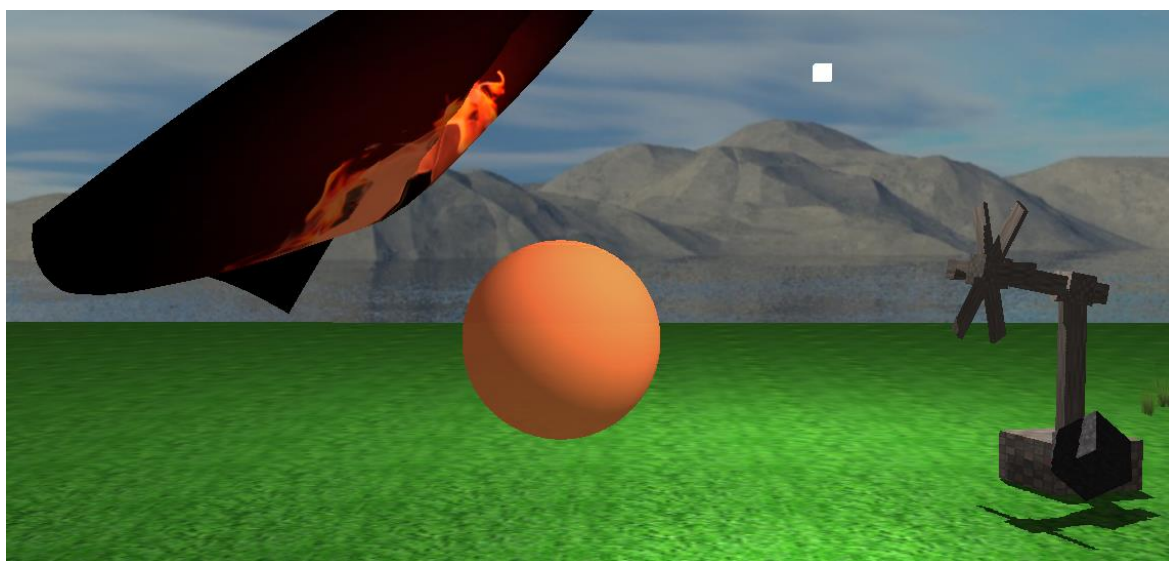


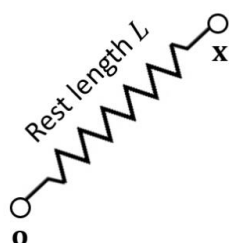
图 6.3.6 : 增大风力（布料被吹起）

7. 布料仿真

7.1 简介

7.1.1 弹簧质点系统

Example: A Spring



Energy:

$$E(\mathbf{x}) = \frac{k}{2} (\|\mathbf{x}\| - L)^2$$

Force:

$$\begin{aligned} \mathbf{f}(\mathbf{x}) &= -\nabla E(\mathbf{x}) = -k(\|\mathbf{x}\| - L) \left(\frac{\partial \|\mathbf{x}\|}{\partial \mathbf{x}} \right)^T \\ &= -k(\|\mathbf{x}\| - L) \frac{\mathbf{x}}{\|\mathbf{x}\|} \end{aligned}$$

Tangent stiffness:

$$\begin{aligned} \mathbf{H}(\mathbf{x}) &= -\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = k \frac{\mathbf{x}\mathbf{x}^T}{\|\mathbf{x}\|^2} + k(\|\mathbf{x}\| - L) \frac{1}{\|\mathbf{x}\|} - k(\|\mathbf{x}\| - L) \frac{\mathbf{x}}{\|\mathbf{x}\|^2} \frac{\mathbf{x}^T}{\|\mathbf{x}\|} \\ &= k \frac{\mathbf{x}\mathbf{x}^T}{\|\mathbf{x}\|^2} + k \left(1 - \frac{L}{\|\mathbf{x}\|} \right) \left(\mathbf{I} - \frac{\mathbf{x}\mathbf{x}^T}{\|\mathbf{x}\|^2} \right) \end{aligned}$$

$$\frac{\partial \|\mathbf{x}\|}{\partial \mathbf{x}} = \frac{\mathbf{x}^T}{\|\mathbf{x}\|}$$

图 7.1.1.1 : 单端弹簧

如上图，设弹簧的一端固定在原点 $\mathbf{o}$ 上， L 是弹簧原长（Rest Length）， $\mathbf{x}$ 是弹簧末端的位置。

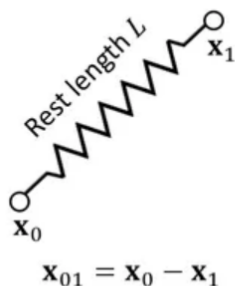
Energy 计算当前弹簧的弹性势能，其中 $(\|\mathbf{o}\| - L)$ 为弹簧的形变量。

弹簧的弹力希望释放掉弹簧的弹性势能， $E(\mathbf{x})$ 对 $\mathbf{x}$ 求负梯度即得弹簧的弹力，上图中的

Force 的结果即 Hooke 定律，其中 $\frac{\mathbf{x}}{\|\mathbf{x}\|}$ 为指向力的方向的单位向量。

Tangent stiffness $\mathbf{H}(\mathbf{x})$ 是 $E(\mathbf{x})$ 的 Hessian 矩阵，其中 stiffness 是弹簧的硬度或劲度，即其中的系数 k 。

Example: A Spring with Two Ends



Energy:

$$E(\mathbf{x}) = \frac{k}{2} (\|\mathbf{x}_{01}\| - L)^2$$

$$\frac{\partial \|\mathbf{x}\|}{\partial \mathbf{x}} = \frac{\mathbf{x}^T}{\|\mathbf{x}\|}$$

Force:

$$\mathbf{f}(\mathbf{x}) = -\nabla E(\mathbf{x}) = \begin{bmatrix} -\nabla_0 E(\mathbf{x}) \\ -\nabla_1 E(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathbf{f}_e \\ -\mathbf{f}_e \end{bmatrix}$$

$$\mathbf{f}_e = -k(\|\mathbf{x}_{01}\| - L) \frac{\mathbf{x}_{01}}{\|\mathbf{x}_{01}\|}$$

Tangent stiffness:

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 E}{\partial \mathbf{x}_0^2} & \frac{\partial^2 E}{\partial \mathbf{x}_0 \partial \mathbf{x}_1} \\ \frac{\partial^2 E}{\partial \mathbf{x}_0 \partial \mathbf{x}_1} & \frac{\partial^2 E}{\partial \mathbf{x}_1^2} \end{bmatrix} = \begin{bmatrix} \mathbf{H}_e & -\mathbf{H}_e \\ -\mathbf{H}_e & \mathbf{H}_e \end{bmatrix}$$

$$\mathbf{H}_e = k \frac{\mathbf{x}_{01} \mathbf{x}_{01}^T}{\|\mathbf{x}_{01}\|^2} + k \left(1 - \frac{L}{\|\mathbf{x}_{01}\|} \right) \left(\mathbf{I} - \frac{\mathbf{x}_{01} \mathbf{x}_{01}^T}{\|\mathbf{x}_{01}\|^2} \right)$$

图 7.1.1.2 : 双端弹簧

若弹簧两端点都可自由移动，则其形变量与两端点的位置都有关。上图中的 $\mathbf{x}_0$ 和 $\mathbf{x}_1$ 是自由移动的两端点。

$E(\mathbf{x})$ 的自变量是 $\mathbf{x} = [\mathbf{x}_0, \mathbf{x}_1]^T$ ，它是一个 6 维向量，包含弹簧两端点的位置坐标。

$\nabla_0 E(\mathbf{x})$ 表示 $E(\mathbf{x})$ 对 $\mathbf{x}_0$ 求梯度， $\nabla_1 E(\mathbf{x})$ 表示 $E(\mathbf{x})$ 对 $\mathbf{x}_1$ 求梯度。

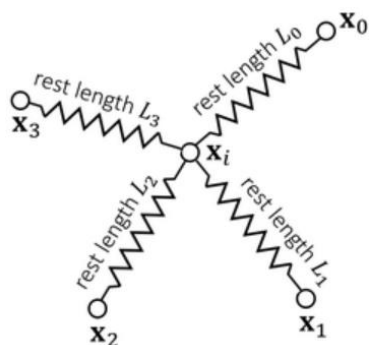
弹簧两端受的弹力等于能量对该位置的负梯度，分别为 $-\nabla_{\mathbf{x}_0} E(\mathbf{x})$ 和 $-\nabla_{\mathbf{x}_1} E(\mathbf{x})$ ，它们大小相同、方向相反。

Tangent stiffness $\mathbf{H}(\mathbf{x})$ 是 $E(\mathbf{x})$ 的 6 x 6 的 Hessian 矩阵，可将两端点的向量分开，

写成分块矩阵的形式。可以证明：对该形式的分块矩阵，若 $\mathbf{H}_e$ 正定，则 $\mathbf{H}(\mathbf{x})$ 半正定。

Multiple Springs

When there are many springs, the energies and the forces can be simply summed up.



$$E = \sum_{j=0}^3 E_j = \sum_{j=0}^3 \left(\frac{1}{2} k (\|\mathbf{x}_i - \mathbf{x}_j\| - L_j)^2 \right)$$

$$\mathbf{f}_i = -\nabla_i E = \sum_{j=0}^3 \left(-k (\|\mathbf{x}_i - \mathbf{x}_j\| - L_j) \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|} \right)$$

图 7.1.1.3 : 多弹簧

若系统由多根弹簧组成，则可分别计算每根弹簧的能量，再对各端点位置求负梯度得到对每个弹簧的分力。对多个弹簧的公共端点，直接叠加各弹簧的分力即可。

7.1.2 用弹簧质点系统模拟布料

Structured Spring Networks

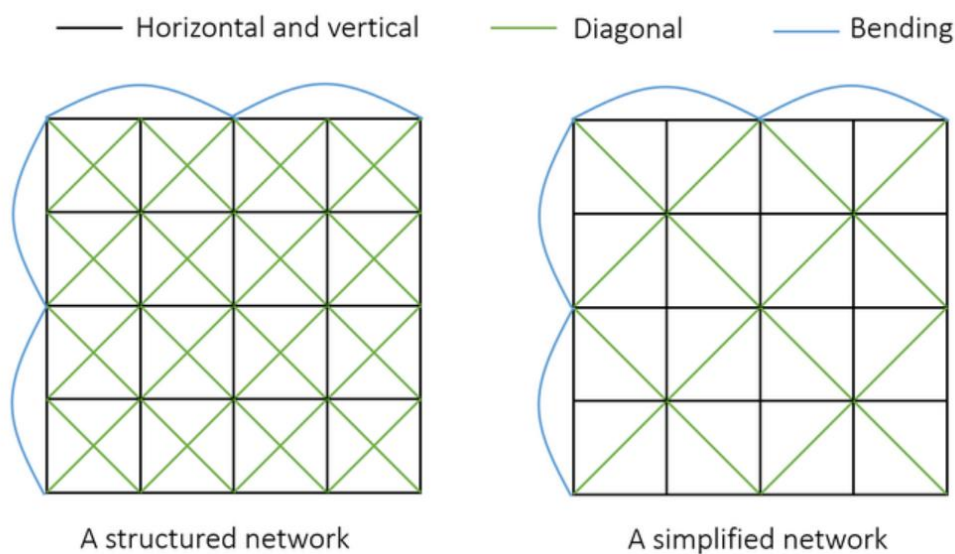


图 7.1.2.1：规则布料的弹簧网

若布料由规则的方形网格构成，则在其上添加如下三种弹簧：

- ① 在网格的边上加弹簧，抵抗布料产生横向和纵向的小褶皱。
- ② 在每个小网格的两条对角线上分别加弹簧，抵抗布料在对角方向上的小褶皱。
- ③ 跨过一个顶点加弹簧，抵抗布料沿长边产生的大翻折（Bending）。

Unstructured Spring Networks

We can also turn an unstructured triangle mesh into a spring network for simulation.

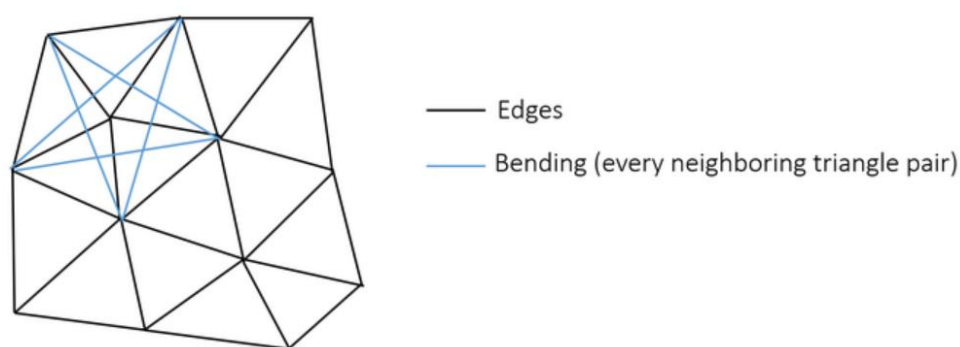


图 7.1.2.2：不规则布料的弹簧网

若布料由不规则的三角网格构成，则在网格的边上加弹簧，跨过内部边（任意两相邻三角形的公共边）加弹簧以抵抗弯曲，即连接两相邻三角形构成的四边形的另外两端点。

下面讨论如何对每个三角形面片添加弹簧。

Triangle Mesh Representation

The basic representation of a triangle mesh uses vertex and triangle lists.

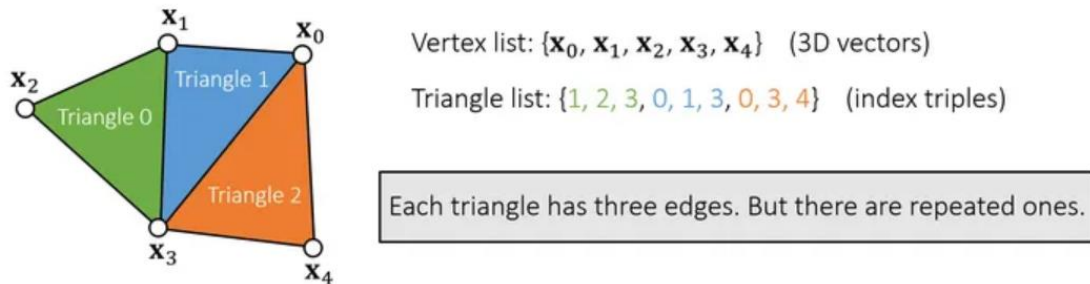


图 7.1.2.3 : 三角面片的存储

如上图，将三角面片存储为如下两组：

- ① 顶点数组：依次存储各顶点的坐标。
- ② 索引数组：存储三角形的顶点在顶点数组中的索引，每 3 个数代表一个三角形。

直接在每条边上加弹簧会导致内部边上重复加弹簧，下面考虑去重。

Topological Construction

The key to topological construction is to sort triangle edge triples.

Each triple contains: edge vertex index 0, edge vertex index 1 and triangle index (index 0 < index).

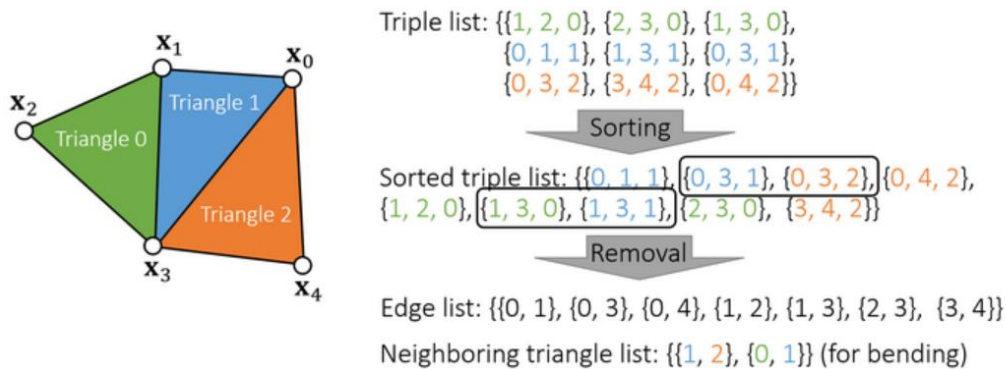


图 7.1.2.4 : 拓扑构造

如上图，从三角面片的数组中提取出所有边构成边数组 Triple list，每条边由 3 个数组成，前两个数表示该边连接的两顶点，第三个数表示该边属于哪个三角形。允许提取得到出现两次的边，但它们的第三个数一定不同。

将 Triple list 按字典序排列，排序后内部边相邻，因为它们的前两个数相同，只有第三个数不同。据此判定内部边，得到去重后的边数组 Edge list，加弹簧即可。具体地，有如下两种情况：

- ① 遍历 Edge list 对三角形的各边加弹簧。
- ② 对内部边，先求该边对应的两三角形的另外两个顶点，再加弹簧。

本节的思路类似于论文 [1] Large Step in Cloth Simulation（见【附件】参考文献），两者的异同：

- (1) 共同点：

考察布料的三种变形：拉伸（stretch）、剪切（shear）、弯曲（bend）。

- (2) 不同点：

- ① 弹簧质点模型从力的角度出发。

- ② Large Step in Cloth Simulation 从能量的角度出发。

本次大作业对布料的调参（忽略单位）：

- ① 结构系数: 1000.0
- ② 剪切系数: 50.0
- ③ 弯曲系数: 400.0

7.1.3 显式 Euler 积分法

Explicit Integration of A Mass-Spring System

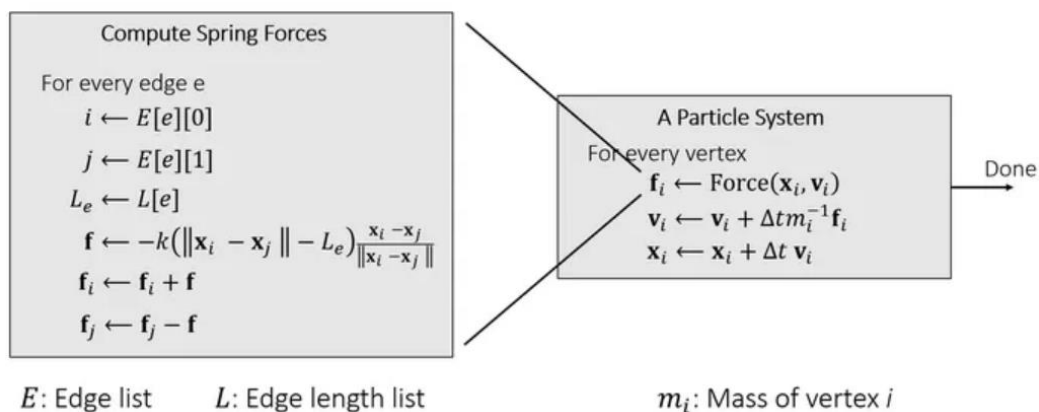


图 7.1.3.1 : 显式 Euler 法

显式 Euler 积分法的计算顺序: 合外力 $\rightarrow$ 加速度 $\rightarrow$ 速度 $\rightarrow$ 位移。具体地: 用显式 Euler 法模拟时, 将布料的所有三角面片的顶点视为一个带弹簧的粒子系统 (Particle System), 先计算所有弹簧对每个顶点的分力, 然后对每个顶点累加合外力, 然后计算加速度, 再更新速度, 最后更新位移。

Explicit Integration of A Mass-Spring System

Explicit integration suffers from numerical instability caused by overshooting, when the stiffness k and/or the time step Δt is too large.

A naive solution is to use a small Δt . But that slows down the simulation.

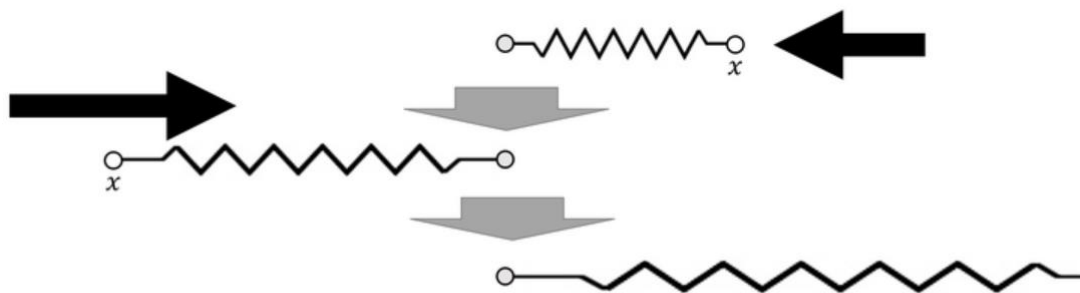


图 7.1.3.2 : overshooting

事实上, 显式 Euler 积分会出现如上图所示的 overshooting 现象。具体地, 弹簧的劲度系数 k 太大或仿真的时间间隔 Δt 太大时, 弹簧的端点可能产生较大的位移, 移动到弹簧的另一端超过原位移的位置, 下一时刻会认为弹簧产生了更大的弹力, 进而产生更大的位移。如此循环往复, 弹簧将迅速振荡出区域。

解决方案:

- (1) 调参: 取合适的 k 值和 Δt 值, 但该方法治标不治本。

本次大作业采用该方法，调参如下：

- ① 弹簧的劲度系数：400.0
- ② 弹簧的阻尼系数：5.0
- ③ 每次渲染循环计算 25 次

(2) 采用隐式 Euler 积分法：将解方程组的问题转化为优化问题，可采用各种优化问题的求解方法进行求解，如 Newton 迭代、Jocabi 法等。

因隐式 Euler 积分法较复杂，非本次大作业的重点，故不再赘述。

论文 [1] Large Step in Cloth Simulation （见 [【附件】 参考文献](#)）采用隐式求解器求解，

7.2 实现

7.2.1 质点类

massPoint.h 实现了点类 Point，点的属性：位置坐标、法向量。代码如下。

```
// 点类
struct Point {
// ===== 成员变量 =====
    glm::vec3 position; // 位置坐标
    glm::vec3 normal; // 法向量

// ===== 构造与析构 =====
    Point() {}
    Point(glm::vec3 _position) : position(_position) {}
};
```

massPoint.h 实现了质点类 MassPoint，质点的属性：质量、位置坐标、法向量、材质坐标、是否固定、力、加速度、速度，并实现了施加力的函数 applyForce() 和显式积分的函数 integrate()，代码如下。

```
// 质点类
struct MassPoint {
// ===== 成员变量 =====
    float mass; // 质量

    glm::vec3 position; // 位置坐标
    glm::vec3 normal; // 法向量
    glm::vec2 textureCoordinate; // 材质坐标

    bool isFixed; // 是否固定

    glm::vec3 force; // 力
    glm::vec3 acceleration; // 加速度
    glm::vec3 velocity; // 速度

// ===== 构造与析构 =====
    MassPoint() : mass(1.0), isFixed(false) {
        force = acceleration = velocity = zero;
    }
};
```

```

    MassPoint(glm::vec3 _position) : mass(1.0), isFixed(false), position(_position) {
        force = acceleration = velocity = zero;
    }

// -----= 成员函数 =-----
// 施加力
void applyForce(glm::vec3 f) {
    force += f;
}

// 显式积分
void integrate(float timeInterval) {
    // 只更新未固定的质点
    if (!isFixed) {
        acceleration = force / mass;
        velocity += acceleration * timeInterval;
        position += velocity * timeInterval;
    }

    // 更新后清空力
    force = zero;
}
};

```

7.2.2 刚体类

rigid.h 中实现了地面类 Ground，将地面视为一个刚体，摩擦系数 0.9。代码如下。

```

// 地面类
struct Ground {
// -----= 成员变量 =-----
    const float friction = 0.9; // 摩擦系数

    int width, height; // 宽度、高度
    glm::vec3 position; // 位置坐标
    glm::vec4 color; // 颜色

    std::vector<Point*> vertices; // 顶点
    std::vector<Point*> faces; // 面

// -----= 构造与析构 =-----
    Ground(glm::vec3 _position, glm::vec2 _size, glm::vec4 _color)
        : position(_position), color(_color) {
        width = _size.x, height = _size.y;

        init();
    }
}

```



```

~Ground() {
    for (int i = 0; i < vertices.size(); i++) delete vertices[i];
    vertices.clear(), faces.clear();
}

// -----= 成员函数 =-----
// 初始化 vertices[] 和 faces[]
void init() {
    vertices.push_back(new Point(glm::vec3(0.0, 0.0, 0.0)));
    vertices.push_back(new Point(glm::vec3(width, 0.0, 0.0)));
    vertices.push_back(new Point(glm::vec3(0.0, 0.0, -height)));
    vertices.push_back(new Point(glm::vec3(width, 0.0, -height)));

    for (int i = 0; i < vertices.size(); i++)
        vertices[i]->normal = glm::vec3(0.0, 1.0, 0.0);

    faces.push_back(vertices[0]);
    faces.push_back(vertices[1]);
    faces.push_back(vertices[2]);
    faces.push_back(vertices[1]);
    faces.push_back(vertices[2]);
    faces.push_back(vertices[3]);

}
};

```

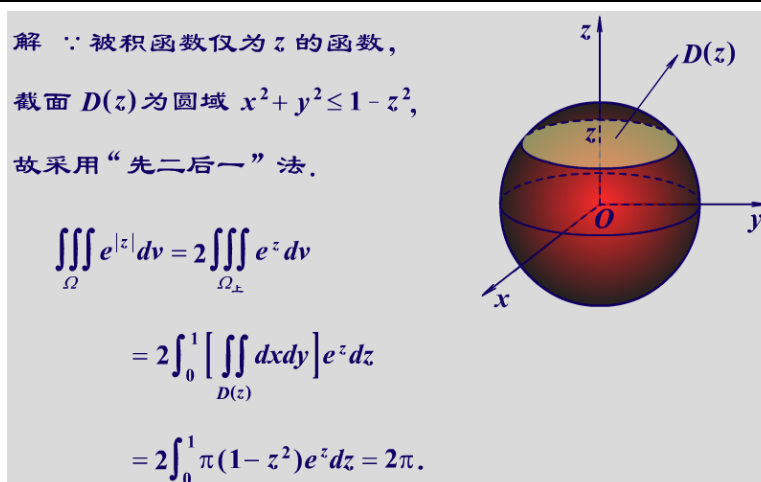


图 7.2.2.1：采样绘制球的原理

为实现球类 `Ball`，`rigid.h` 中先抽象一个刚体球类 `Sphere.h`，采用采样的方式绘制球，其原理类似于如上图所示的对球的三重积分，即先取一个截面，绘制该截面的圆后，再枚举高度绘制其它界面。

实现见下方代码的 `init()` 函数，思路：先根据所取的横截面个数 `parallelCnt` 计算得相邻横截面间的距离 `cycleInterval`，再根据所取的纵截面个数 `meridianCnt` 计算得相邻纵截面间的弧度差 `radianInterval`。枚举截面，从该截面的圆心开始枚举弧度差，预处理出每个三角面片。注意单独绘制球的最高点和最低点所在的面片，因为这两个点处的法向量分别是向上和向下，没有其它向四周的方向。预处理出每个三角面片后，用 `getSphereNormal()` 函数计算处各质点处的法向量，法向量指向球的外侧。

```
// 球类(构造)
```



```

struct Sphere {
// ===== 成员变量 =====
    const int parallelCnt = 250, meridianCnt = 24; // 横竖方向的采样点数

    float radius; // 半径

    std::vector<Point*> vertices; // 顶点
    std::vector<Point*> faces; // 面

// ===== 构造与析构 =====
    Sphere(float _radius) : radius(_radius) {
        init();
    }

    ~Sphere() {
        for (int i = 0; i < vertices.size(); i++) delete vertices[i];
        vertices.clear(), faces.clear();
    }

// ===== 成员函数 =====
    // 获取球的最高点
    Point* getTopPoint() {
        return vertices[0];
    }

    // 获取球的最低点
    Point* getBottomPoint() {
        return vertices.back();
    }

    // 获取下标 (x, y) 处的点
    Point* getPoint(int x, int y) {
        if (x < 0 || x >= parallelCnt || y < 0 || y >= meridianCnt) {
            std::cout << "Point index out of range." << std::endl;
            exit(-1);
        }

        return vertices[1 + x * meridianCnt + y];
    }

    // 求三个顶点 v1, v2, v3 构成的面的法向量
    glm::vec3 getFaceNormal(Point* v1, Point* v2, Point* v3) {
        return cross(v2->position - v1->position, v3->position - v1->position);
    }

    // 求球的各面的法向量
    void getSphereNormal() {
        glm::vec3 normal(0.0, 0.0, 0.0);
    }
}

```

```

    for (int i = 0; i < vertices.size(); i++) vertices[i]->normal = normal;

    // 顶部截面和底部截面的圆的面的法向量后续特别计算
    for (int i = 0; i < faces.size() / 3; i++) {
        Point* v1 = faces[i * 3 + 0];
        Point* v2 = faces[i * 3 + 1];
        Point* v3 = faces[i * 3 + 2];

        normal = getFaceNormal(v1, v3, v2);
        v1->normal += normal;
        v2->normal += normal;
        v3->normal += normal;
    }

    for (int i = 0; i < vertices.size(); i++)
        vertices[i]->normal = glm::normalize(vertices[i]->normal);
}

// 初始化 vertices[] 和 faces[]
void init() {
    // 精度
    double cycleInterval = radius * 2.0 / (parallelCnt + 1); // 截面距离
    double radianInterval = 2.0 * M_PI / meridianCnt; // 弧度差

    // 最高点
    glm::vec3 pos(0.0, radius, 0.0);
    vertices.push_back(new Point(pos));

    // 中间点
    for (int i = 0; i < parallelCnt; i++) {
        pos.y -= cycleInterval;

        for (int j = 0; j < meridianCnt; j++) {
            double xzLen = radius * sqrt(1.0 - pow(pos.y / radius, 2));
            double xRadian = j * radianInterval; // 在 xz 平面的投影的弧长
            pos.x = xzLen * sin(xRadian), pos.z = xzLen * cos(xRadian); // 圆上的点的坐标
            vertices.push_back(new Point(pos));
        }
    }

    // 最低点
    pos = glm::vec3(0.0, -radius, 0.0);
    vertices.push_back(new Point(pos));

    // 截面
    // 顶部截面
    for (int i = 0; i < meridianCnt; i++) {
        // * //

```

```

        //  \  //
        // *---* //
        faces.push_back(getPoint(0, i));
        faces.push_back(getTopPoint());
        faces.push_back(getPoint(0, (i + 1) % meridianCnt));
    }
    // 中间截面
    for (int i = 0; i < parallelCnt - 1; i++) {
        for (int j = 0; j < meridianCnt; j++) {
            //  *--* //
            //  | /  //
            //  *    //
            faces.push_back(getPoint(i, j));
            faces.push_back(getPoint(i, (j + 1) % meridianCnt));
            faces.push_back(getPoint(i + 1, j));

            //      * //
            //    / | //
            //  *--* //
            faces.push_back(getPoint(i + 1, (j + 1) % meridianCnt));
            faces.push_back(getPoint(i + 1, j));
            faces.push_back(getPoint(i, (j + 1) % meridianCnt));
        }
    }
    // 底部截面
    for (int i = 0; i < meridianCnt; i++) {
        // *---* //
        //  \ /  //
        //  *    //
        faces.push_back(getBottomPoint());
        faces.push_back(getPoint(parallelCnt - 1, i));
        faces.push_back(getPoint(parallelCnt - 1, (i + 1) % meridianCnt));
    }

    // 求各面的法向量
    getSphereNormal();
}
};

```

基于 Sphere 类实现球类 Ball，球的属性：摩擦系数 0.8、球心、半径、颜色。

```

// 球类
struct Ball {
    const float friction = 0.8; // 摩擦系数

    glm::vec3 center; // 球心
    float radius; // 半径
    glm::vec4 color; // 颜色
    Sphere *sphere; // 球

```

```

    Ball(glm::vec3 _center, float _radius, glm::vec4 _color)
        : center(_center), radius(_radius), color(_color) {
        sphere = new Sphere(radius);
    }
};

```

7.2.3 弹簧类

spring.h 中实现了弹簧类 Spring，弹簧的主要属性：两 endpoints（质点）、原长、劲度系数、阻尼系数，实现了用于计算弹簧内力的函数 applyInternalForce()。代码如下。

```

struct Spring {
// -----= 成员变量 =-----
    MassPoint *point1, *point2; // 弹簧的两端点

    float originalLength; // 原长
    float hookCoefficient; // 劲度系数
    float dampCoefficient; // 阻尼系数

// -----= 构造与析构 =-----
    Spring(MassPoint *_point1, MassPoint *_point2, float _hookCoefficient)
        : point1(_point1), point2(_point2),
        hookCoefficient(_hookCoefficient), dampCoefficient(5.0) {
        glm::vec3 currentVector = point2->position - point1->position;
        originalLength = sqrt(dot(currentVector, currentVector));
    }

// -----= 成员函数 =-----
    // 计算内力
    void applyInternalForce(float timeInterval) {
        glm::vec3 currentVector = point2->position - point1->position;
        float currentLength = sqrt(dot(currentVector, currentVector));

        glm::vec3 fDirection = (point2->position - point1->position) / currentLength;
        glm::vec3 vDelta = point2->velocity - point1->velocity;
        glm::vec3 f = fDirection * ((currentLength - originalLength) * hookCoefficient
            + dot(vDelta, fDirection) * dampCoefficient);
        point1->applyForce(f), point2->applyForce(-f);
    }
};

```

7.2.4 布料类

cloth.h 中实现了布料类 Cloth，布料的主要属性：结构系数、剪切系数、弯曲系数、位置坐标，还有用于控制绘制模式的枚举变量 DrawMode，以支持绘制点模式、线模式、面模式。此外，为将布料固定在空中，取两个钉子节点作为固定点，即设定质点的 isFixed 属性为 true，更新受力时只

对 isFixed 属性为 false 的质点更新。代码如下。

```
// 布料类
struct Cloth {
// ===== 成员变量 =====
    const int pointsDensity = 4; // 相邻两质点的间隔
    const int iterationFrequency = 25; // 帧率
    const double structuralCoefficient = 1000.0; // 结构系数
    const double shearCoefficient = 50.0; // 剪切系数
    const double bendingCoefficient = 400.0; // 弯曲系数

    enum DrawMode {
        DRAW_NODES, // 绘制点
        DRAW_LINES, // 绘制线
        DRAW_FACES // 绘制面
    };
    DrawMode drawMode = DRAW_FACES; // 绘制模式

    float width, height; // 宽度, 高度
    int pointsPerRow, pointsPerColumn; // 每行、每列的质点数

    glm::vec3 position; // 位置坐标

    std::vector<MassPoint*> points; // 质点
    std::vector<Spring*> springs; // 弹簧
    std::vector<MassPoint*> faces; // 面

    glm::vec2 pin1, pin2; // 固定布料的两枚钉子

// ===== 构造与析构 =====
    Cloth(glm::vec3 _position, glm::vec2 _size) : position(_position) {
        width = _size.x, height = _size.y;

        init();
    }

    ~Cloth() {
        for (int i = 0; i < points.size(); i++) delete points[i];
        for (int i = 0; i < springs.size(); i++) delete springs[i];

        points.clear(), springs.clear(), faces.clear();
    }

// ===== 成员函数 =====
    ...
}
```

布料用采样的方式绘制，上述代码中的成员变量 pointsDensity 、 iterationFrequency 、 pointsPerRow 、 pointsPerColumn 是控制采样的参数。下面代码中的成员函数 init() 用采样预处理出布料的各顶点、弹簧位置、三角面片，并取布料的左上角和右上角作钉子。

```

// 求下标 (x, y) 处的质点
MassPoint* getNode(int x, int y) {
    return points[y * pointsPerRow + x];
}

// 初始化成员变量
void init() {
    pointsPerRow = width * pointsDensity;
    pointsPerColumn = height * pointsDensity;

    // 生成质点
    for (int i = 0; i < pointsPerRow; i++) {
        for (int j = 0; j < pointsPerColumn; j++) {
            MassPoint *point = new MassPoint(glm::vec3((double)j / pointsDensity, -((double)i /
pointsDensity), 0));

            // 设置材质坐标
            point->textureCoordinate.x = (double)j / (pointsPerRow - 1);
            point->textureCoordinate.y = (double)i / (1 - pointsPerColumn);

            points.push_back(point);
        }
    }

    // 生成弹簧
    for (int i = 0; i < pointsPerRow; i++) {
        for (int j = 0; j < pointsPerColumn; j++) {
            // 结构系数
            if (i < pointsPerRow - 1)
                springs.push_back(new Spring(getNode(i, j), getNode(i + 1, j),
structuralCoefficient));
            if (j < pointsPerColumn - 1)
                springs.push_back(new Spring(getNode(i, j), getNode(i, j + 1),
structuralCoefficient));

            // 剪切系数
            if (i < pointsPerRow - 1 && j < pointsPerColumn - 1) {
                springs.push_back(new Spring(getNode(i, j), getNode(i + 1, j + 1),
shearCoefficient));
                springs.push_back(new Spring(getNode(i + 1, j), getNode(i, j + 1),
shearCoefficient));
            }

            // 弯曲系数
            if (i < pointsPerRow - 2)
                springs.push_back(new Spring(getNode(i, j), getNode(i + 2, j),
bendingCoefficient));
            if (j < pointsPerColumn - 2)

```

```

        springs.push_back(new Spring(getNode(i, j), getNode(i, j + 2),
bendingCoefficient));
    }
}

// 取布料上边的两顶点作为固定点
pin1 = glm::vec2(0, 0), pin2 = glm::vec2(pointsPerRow - 1, 0);
// 固定点
pin(pin1, glm::vec3(1.0, 0.0, 0.0));
pin(pin2, glm::vec3(-1.0, 0.0, 0.0));

// 三角面片
for (int i = 0; i < pointsPerRow - 1; i++) {
    for (int j = 0; j < pointsPerColumn - 1; j++) {
        // *--* //
        // | / //
        // * //

        faces.push_back(getNode(i + 1, j));
        faces.push_back(getNode(i, j));
        faces.push_back(getNode(i, j + 1));

        // * //
        // / | //
        // *--* //

        faces.push_back(getNode(i + 1, j + 1));
        faces.push_back(getNode(i + 1, j));
        faces.push_back(getNode(i, j + 1));
    }
}
}

```

上述代码中的 `pin()` 函数用于固定顶点，此外还实现了 `unpin()` 函数用于取消顶点的固定，代码如下。

```

// 给下标 index 的质点的位置加一个偏移量后固定
void pin(glm::vec2 index, glm::vec3 offset) {
    if (index.x < 0 || index.x >= pointsPerRow || index.y < 0 || index.y >= pointsPerColumn) return;

    getNode(index.x, index.y)->position += offset;
    getNode(index.x, index.y)->isFixed = true;
}

// 取消下标 index 的质点的固定
void unpin(glm::vec2 index) {
    if (index.x < 0 || index.x >= pointsPerRow || index.y < 0 || index.y >= pointsPerColumn) return;

    getNode(index.x, index.y)->isFixed = false;
}

```

实现了用于求布料各面的法向量的函数 `getNormal()`，代码如下。

```

// 求三个顶点 point1, point2, point3 构成的面的法向量
glm::vec3 getFaceNormal(MassPoint *point1, MassPoint *point2, MassPoint *point3) {
    return cross(point2->position - point1->position, point3->position - point1->position);
}

// 求各面的法向量
void getNormal() {
    glm::vec3 normal(0.0, 0.0, 0.0);
    for (int i = 0; i < points.size(); i++) points[i]->normal = normal;

    for (int i = 0; i < faces.size() / 3; i++) {
        MassPoint *point1 = faces[3 * i + 0];
        MassPoint *point2 = faces[3 * i + 1];
        MassPoint *point3 = faces[3 * i + 2];

        normal = getFaceNormal(point1, point2, point3);
        point1->normal += normal;
        point2->normal += normal;
        point3->normal += normal;
    }

    for (int i = 0; i < points.size(); i++)
        points[i]->normal = glm::normalize(points[i]->normal);
}

```

实现了用于计算质点受力的各函数，代码如下。

```

// 施加力
void applyForce(glm::vec3 force) {
    for (int i = 0; i < points.size(); i++) points[i]->applyForce(force);
}

// 计算力
void getForce(float timeInterval, glm::vec3 gravity) {
    // 质点受重力
    for (int i = 0; i < points.size(); i++)
        points[i]->applyForce(gravity * points[i]->mass);

    // 弹簧受内力
    for (int i = 0; i < springs.size(); i++)
        springs[i]->applyInternalForce(timeInterval);
}

// 显式积分
void integrate(float timeInterval) {
    for (int i = 0; i < points.size(); i++)
        points[i]->integrate(timeInterval);
}

```

实现了布料与地面、球的碰撞检测，具体地：

- ① 布料与地面的碰撞：若质点的 y 坐标 $<$ 地面的 y 坐标，则认为发生碰撞，将该质点

的速度乘上地面的摩擦系数以实现减速。

② 布料与球的碰撞：若质点 P 与球心 O 的距离小于安全距离（本次大作业取球半径的 1.05 倍），则认为发生碰撞，将该质点沿向量 OP 方向平移一段距离，并将该质点的速度乘上球的摩擦系数以实现减速。

```
// 求质点在世界坐标中的坐标
glm::vec3 getWorldPosition(MassPoint *point) {
    return position + point->position;
}

// 设置质点在世界坐标中的坐标
void setWorldPosition(MassPoint *point, glm::vec3 pos) {
    point->position = pos - position;
}

// 碰撞检测
void collisionResponse(Ground *ground, Ball *ball) {
    for (int i = 0; i < points.size(); i++) {
        // 布料与地面的碰撞
        if (getWorldPosition(points[i]).y < ground->position.y) {
            points[i]->position.y = ground->position.y - position.y + 0.01; // + 0.01 避免深度冲突

            points[i]->velocity *= ground->friction; // 布料受地面的摩擦力
        }

        // 布料与球的碰撞
        glm::vec3 distanceVector = getWorldPosition(points[i]) - ball->center;
        float distance = sqrt(dot(distanceVector, distanceVector));
        float safeDistance = ball->radius * 1.05;
        if (distance < safeDistance) {
            if (distance >= 0.0001) distanceVector = glm::normalize(distanceVector);
            setWorldPosition(points[i], distanceVector * safeDistance + ball->center);
            points[i]->velocity *= ball->friction; // 布料受球的摩擦力
        }
    }
}
```

7.2.5 渲染类

render.h 对各种物体的绘制操作进行封装，如读取 shader、设置 VAO 和 VBO、设置属性指针、设置 uniform 变量等操作。

render.h 中包含的渲染类：

- ① RigidRender：刚体渲染父类。
- ② GroundRender：地面渲染类。
- ③ BallRender：球渲染类。
- ④ ClothRender：布料渲染类。
- ⑤ SpringRender：弹簧渲染类，用于绘制布料的线绘制模式时绘制弹簧（线）。
- ⑥ ClothSpringRender：布料与弹簧渲染类，将布料的渲染与弹簧的渲染联合。

渲染类的实现与平时实验基本相同，因代码量较大，此处不展示代码。

7.3 效果

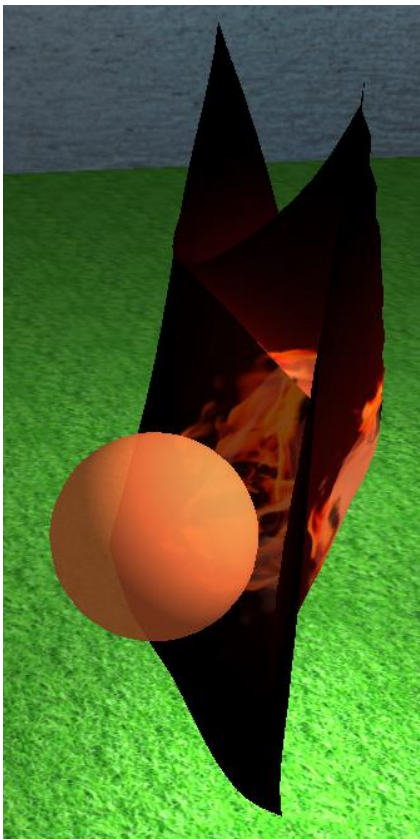


图 7.3.1：无风时布料受重力，与球和地面碰撞（面绘制模式）

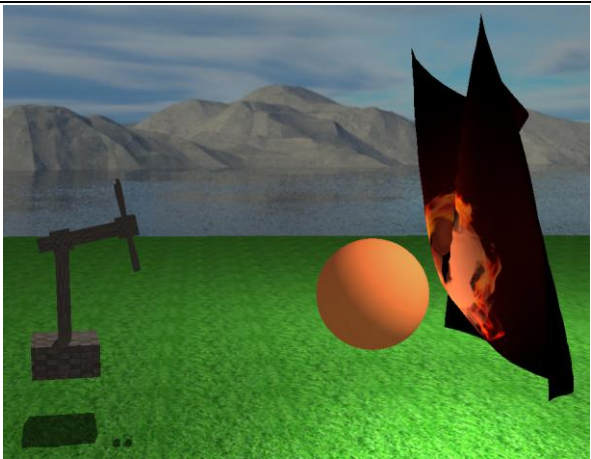


图 7.3.2：风车吹起布料

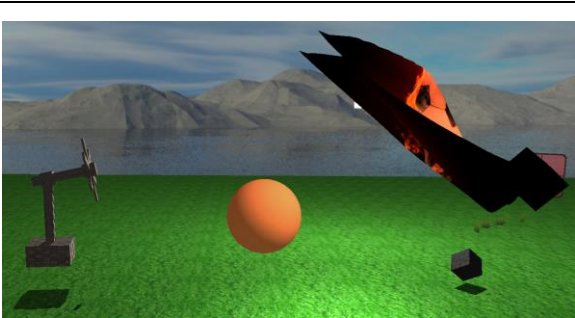


图 7.3.3：加大风力



图 7.3.4 : 以其他角度吹布料（风车视角）

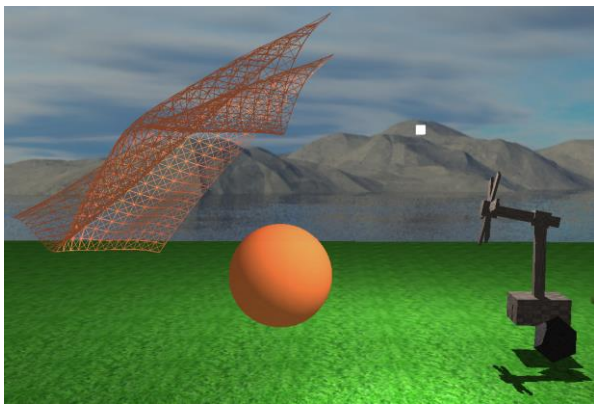


图 7.3.5 : 线绘制模式

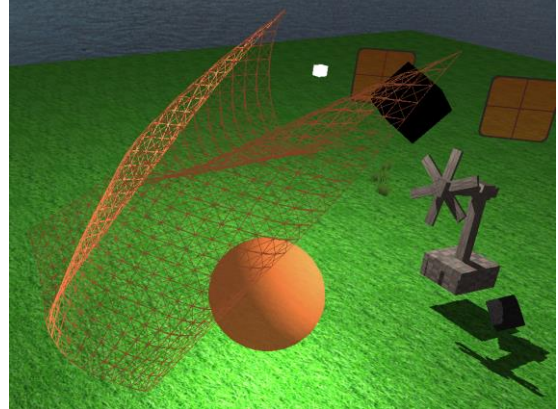


图 7.3.6 : 观察到布料由方形网格构成

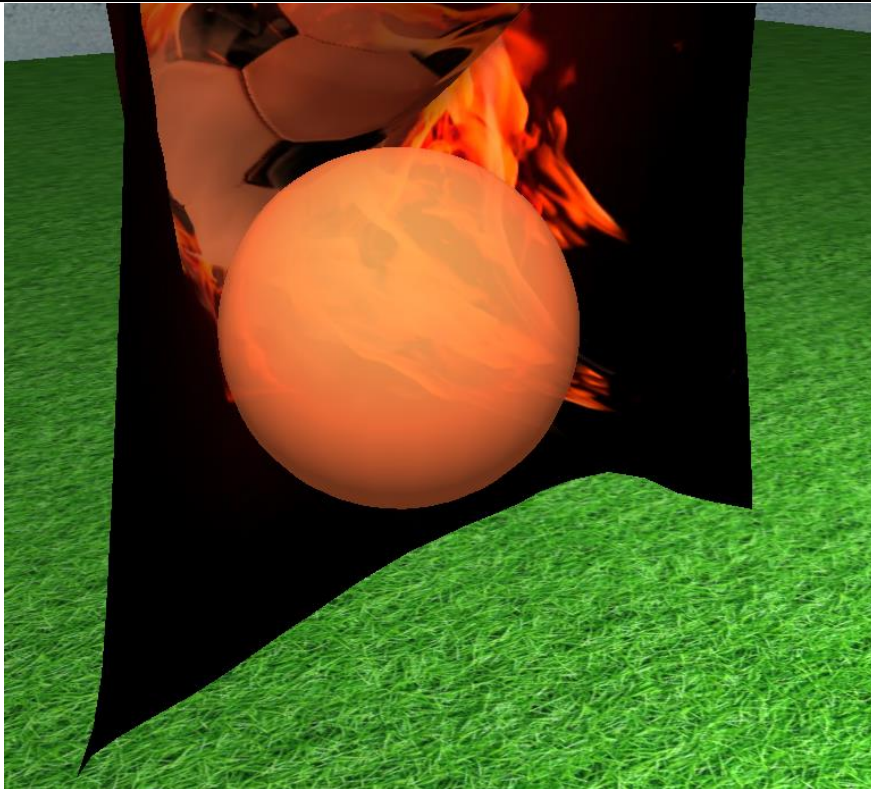


图 7.3.7 : 半透明的球

8. Blinn-Phong 光照

8.1 简介

Phong 光照在镜面反射时可能出问题。具体地，物体反光度很低时，可能出现大片粗糙的高光区域。



图 8.1.1：高光断层

上图是一个反光度为 1.0 的地面，观察到镜面高光的边缘部分出现明显断层。

出现高光断层现象是因为观察向量与反射向量间的夹角大于 90° 时，点积结构为负，镜面光分量变为 0.0。对漫反射，光线与视线夹角大于 90° 时不收到光线是合理的，因为法线与光源夹角大于 90° 时，光源处于被照表面的下方，此时漫反射光分量为 0.0。

上述情况不适用于镜面光分量。如下图所示，考察镜面光时测量的角度不是光源与法线的夹角，而是视线与反射光线的夹角。下图左图展示 Phong 光照中的反射向量 $\vec{R}$ ，其中 $\theta < 90^\circ$ ；下图右图中 $\theta > 90^\circ$ ，此时镜面光分量变为 0.0。该现象在大多数情况中不出现问题，因为观察方向离反射方向很远。但当物体反光度很小时，其产生的镜面高光半径足以让相反方向的光线对亮度产生足够大的影响，此时不能忽略它们对镜面光分量的贡献。

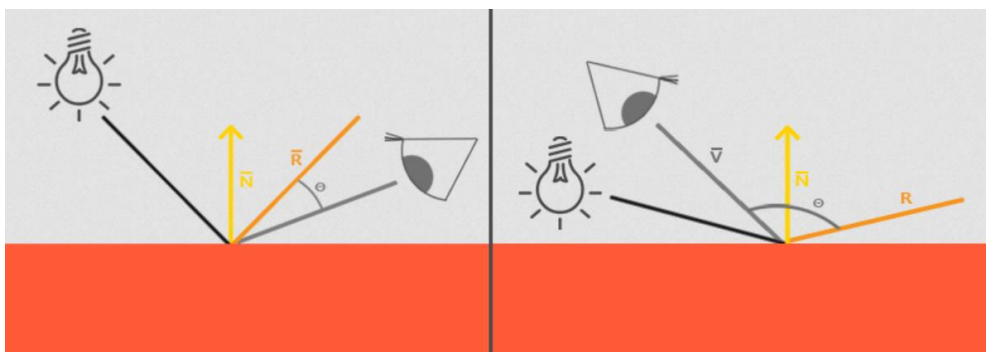


图 8.1.2：镜面光示意图

产生上述问题是因为 Phong 光照依赖于反射向量。Blinn-Phong 光照与 Phong 光照对环境光和漫反射的处理相同，在对镜面光的处理上，前者不再依赖反射向量，而引入**半程向量**（Halfway Vector），它是光线与视线夹角一半方向上的一个单位向量，如下图所示。当半程向量与法线向量越接近时，镜面光分量越大。视线与（现在不需要的）反射向量重合时，半程向量与法线重合，故观察者视线越接近于原本反射光线的方向时，镜面高光增强。此时，无论观察者向哪个方向看，半程向量与表面法线间的夹角都不超过 90° （除非光源在表面以下）。

Blinn-Phong 光照与 Phong 光照产生的效果稍有不同，但大部分时候前者更自然，特别是低高光区域。

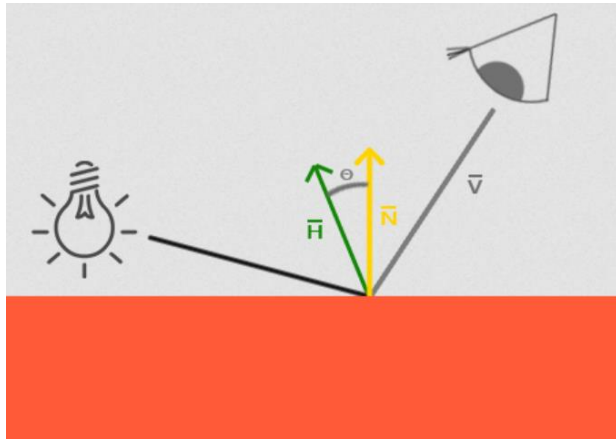


图 8.1.3 : 半程向量示意图

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}$$

半程向量的计算：，其中 $\vec{L}$ 为光线的方向向量， $\vec{V}$ 为观察向量。

引入半程向量后，高光断层消失。下图对比了 Phong 光照与 Blinn-Phong 光照在镜面光分量为 0.5 时的效果。



图 8.1.4 : Phong 光照与 Blinn-Phong 光照在镜面光分量为 0.5 时的效果对比

8.2 实现

在用于绘制带阴影的物体的 colors.fshader 中，镜面光分量的计算代码如下。

```
// 镜面反射
vec3 viewDir = normalize(viewPos - fs_in.FragPos);
vec3 reflectDir = reflect(-lightDir, normal);
float spec = 0.0;
if (blinn) { // Blinn-Phong 模型
    vec3 halfwayDir = normalize(lightDir + viewDir);
    spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0);
}
else {
    vec3 reflectDir = reflect(-lightDir, normal);
    spec = pow(max(dot(viewDir, reflectDir), 0.0), 8.0);
}
vec3 specular = spec * lightColor;
```

8.3 效果



图 8.3.1 : Phong 光照

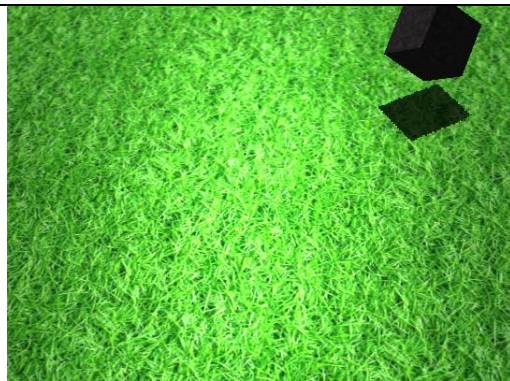


图 8.3.2 : Blinn-Phong 光照

9. Gamma 校正

9.1 简介

过去大多数显示器是**阴极射线管显示器**（CRT），它输入电压产生约为其 2.2 次幂的亮度，这个值称为**显示器 Gamma**。显示器的非线性映射可让亮度在人类眼中看起来更好，但渲染图像时会产生问题：配置的亮度和颜色是基于显示器显示的，这样所有的配置实际上是非线性的。如下图，点线代表线性颜色值（即 $\text{Gamma} = 1$ 的理想状态），实线代表显示器显示的颜色。观察到：显示器显示的亮度与线性映射的最大、最小亮度相同，只有中间的亮度会被压暗。

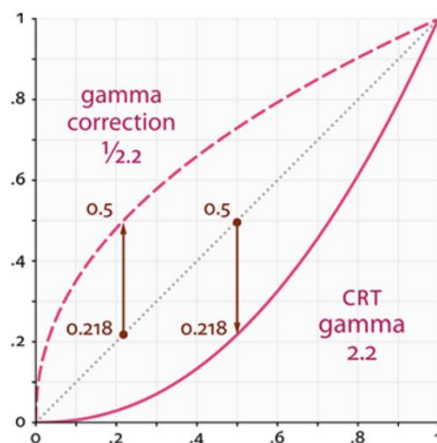


图 9.1.1：显示器的非线性映射

若在线性空间（ $\text{Gamma} = 1$ ）中配置颜色，则配置的所有颜色和光照变量在物理上是错误的，在显示器上也很少能正确显示，该现象在使用高级的光照算法会更加明显，如下图所示。

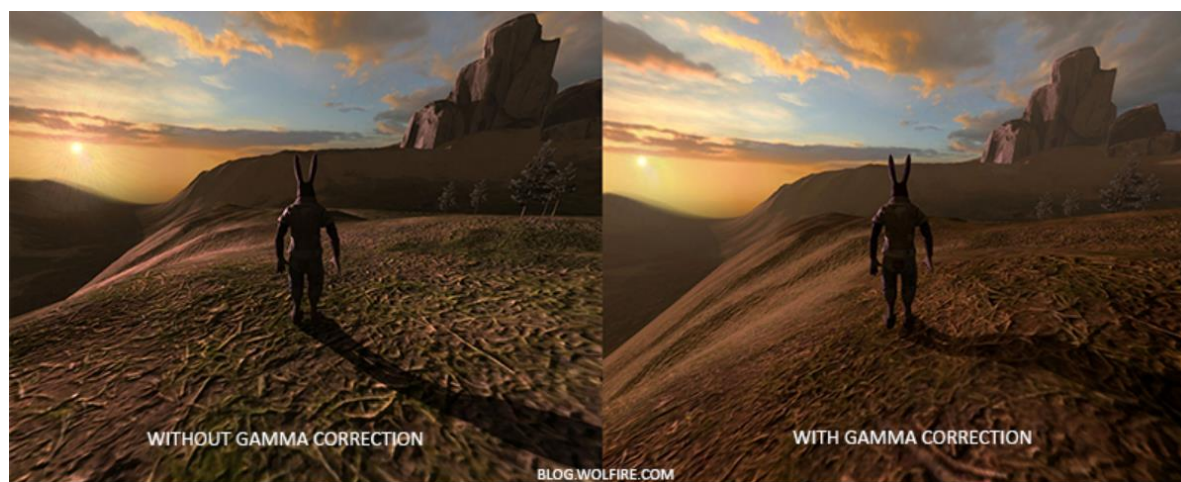


图 9.1.2：有无 Gamma 校正的对比

因此，设计师常将光照值设置得比原本更亮一些，因为显示器会将其显示得更暗一些。

Gamma 校正（Gamma Correction）的思路：在最终颜色输出上应用显示器 Gamma 的倒数。如 图 9.1.1 的虚线是显示器 Gamma 曲线的翻转曲线，若在颜色输出前应用该反转曲线，则可让最终颜色变为线性映射，得到的中间色调会更亮，即平衡了显示器的压暗。

9.2 实现

在 colors.fshader 中添加 Gamma 校正，本次大作业取 $\text{gamma} = 1.2$ ，代码如下。

```
FragColor = vec4(lightning, 1.0);

// Gamma 校正
if (gamma) {
    float Gamma = 1.2f;
    FragColor.rgb = pow(FragColor.rgb, vec3(1.0 / Gamma));
}
```

9.3 效果

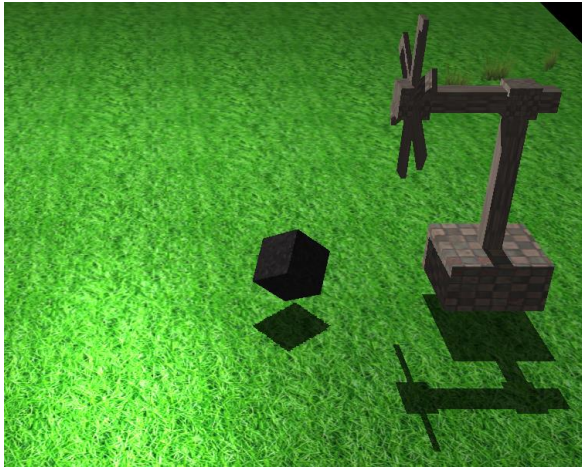


图 9.3.1 : 关闭 Gamma 校正

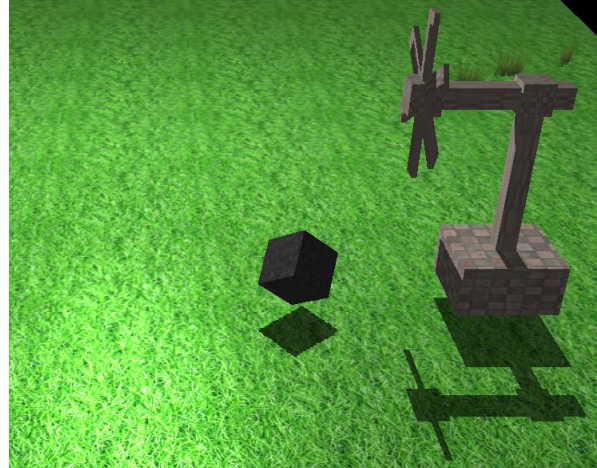


图 9.3.2 : 开启 Gamma 校正

10. 使用说明书

10.1 摄像机

10.1.1 模式切换

输入	功能
键盘 1	切换到自由摄像机模式
键盘 2	切换到风车模式
键盘 3	切换到自动环绕摄像机模式

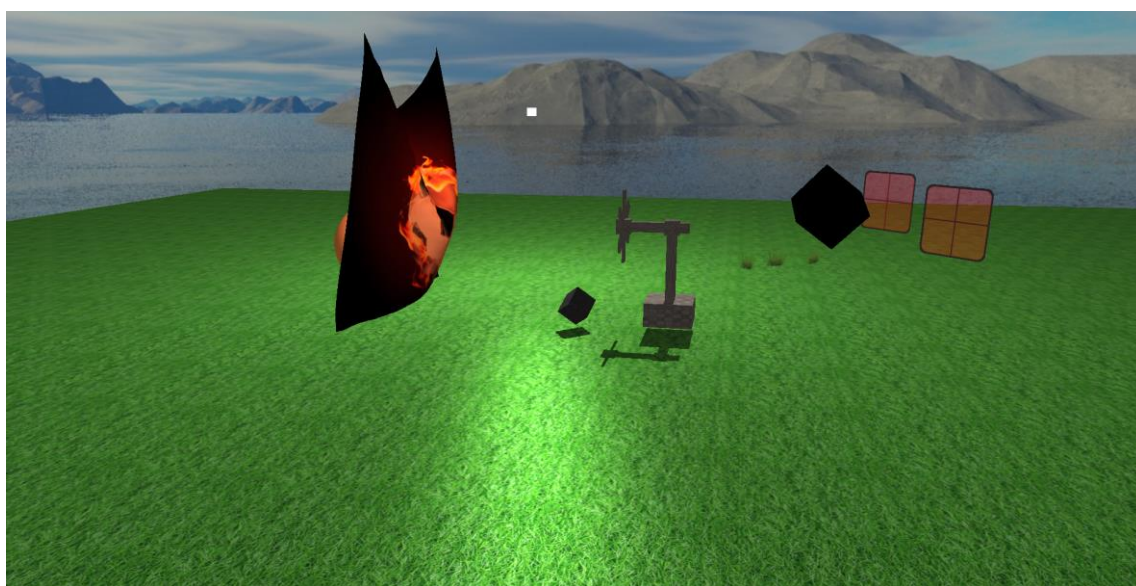


图 10.1.1.1 ： 自由摄像机模式

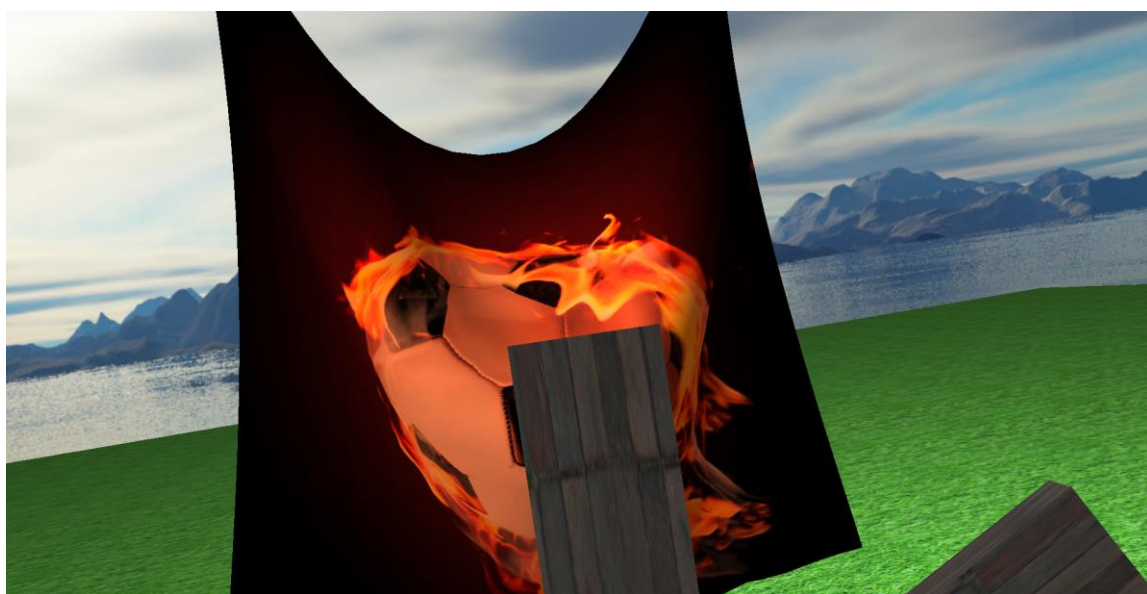


图 10.1.1.2 ： 风车模式

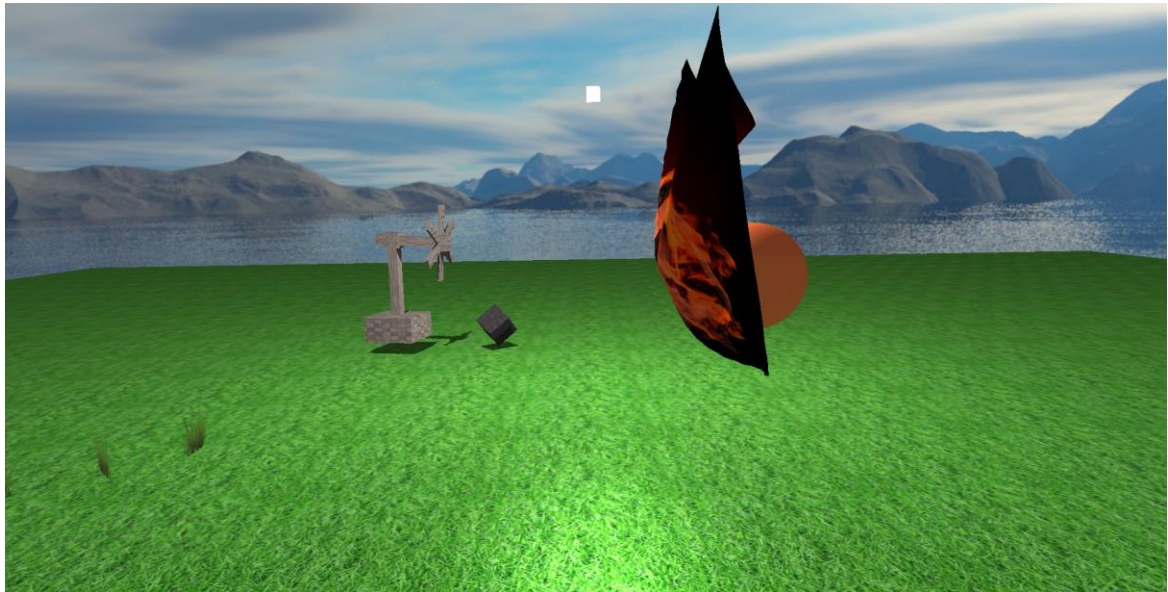


图 10.1.1.3 ：自由环绕模式（运行时摄像机会环绕场景移动）

10.1.2 自由摄像机模式

输入	功能
键盘 W、A、S、D	摄像机前、后、左、右移动
鼠标 移动	移动摄像机视角
鼠标 滚轮	调整摄像机焦距

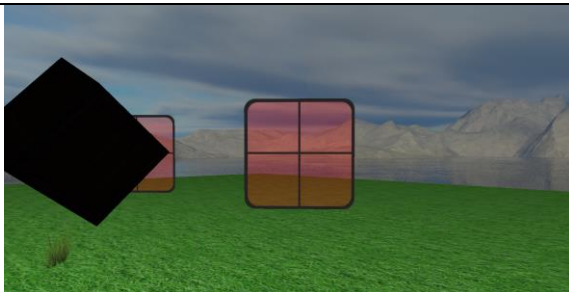


图 10.1.2.1 ：焦距（大）



图 10.1.2.2 ：焦距（小）

10.1.3 风车模式

输入	功能
键盘 W、A、S、D	风车前、后、左、右移动
鼠标 移动	移动风车视角

10.1.4 自动环绕摄像机模式

输入	功能
键盘 W、S	摄像机高度上、下移动

10.2 手电筒

输入	功能
键盘 T	开启或关闭手电筒

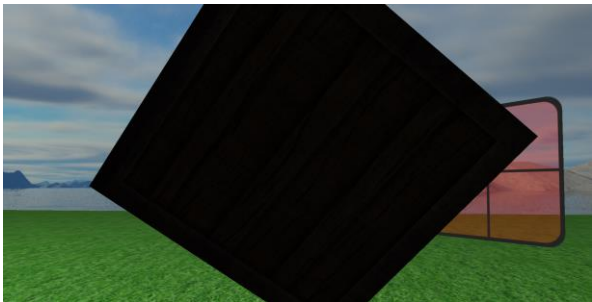


图 10.2.1 ： 关闭手电筒

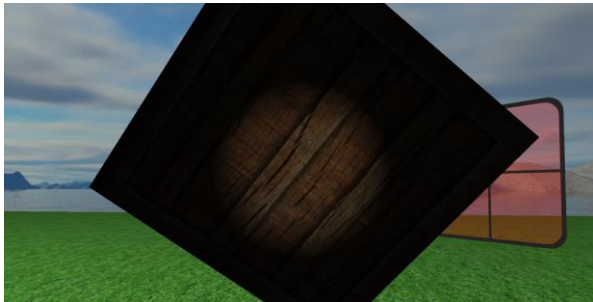


图 10.2.2 ： 开启手电筒

10.3 光影

输入	功能
键盘 B	切换 Phong 光照与 Blinn-Phong 光照
键盘 P	开启或关闭 PCF
键盘 G	开启或关闭 Gamma 校正

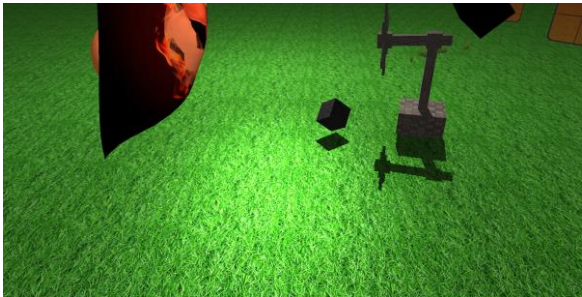


图 10.3.1 ： Phong 光照

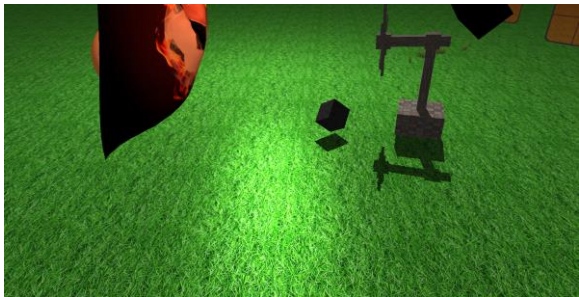


图 10.3.2 ： Blinn-Phong 光照



图 10.3.3 ： 关闭 PCF



图 10.3.4 ： 开启 PCF

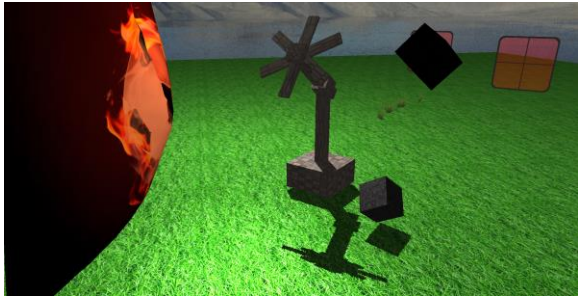


图 10.3.5：关闭 Gamma 校正

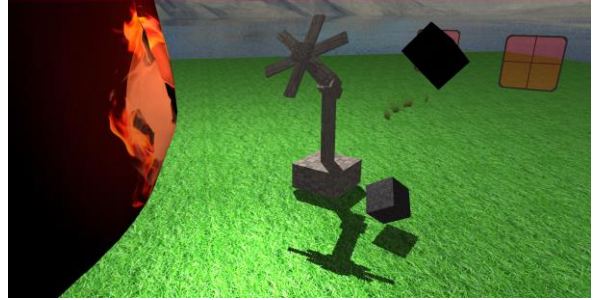


图 10.3.6：开启 Gamma 校正

10.4 风车

输入	功能
键盘 C	开启或关闭风车
键盘 - / +	减小或增大风车转速

静态图片无法显示效果，请参考 [【附件】 视频演示](#)。

10.5 布料仿真

输入	功能
键盘 L	开启或停止仿真
键盘 F	切换布料的面绘制模式与线绘制模式
键盘 ↑、↓、←、→	刮对应方向的风
键盘 9、0	取消布料左上角/右上角的固定



图 10.5.1：布料的面绘制模式



图 10.5.2：布料的线绘制模式

11. 总结

本部分将指出本次大作业中每个部分分别符合何要求。

11.1 基本要求

- (1) 包含地板的室内或室外场景：天空盒、草地
- (2) 至少 4 层的层级建模：风车（6 层）
- (3) 至少四层的层级建模：草坪、草、窗户、钢框箱子（含镜面光贴图）、石头、风车、布料
- (4) 光照：Phong 光照、Blinn-Phong 光照
- (5) 材质：钢框箱子
- (6) 阴影：阴影映射
- (7) 交互控制视角切换：自由摄像机模式、风车模式、自动环绕摄像机模式
- (8) 交互控制至少两个物体：移动风车、开启与关闭风车、风车吹布料、刮风

11.2 Shining Points

- (1) 透明材质：草、玻璃
- (2) 多光源：点光源（光源正方体）、平行光、聚光（手电筒）
- (3) 复杂建模：布料仿真
- (4) 复杂动画：风车吹布料、布料与球和地面的碰撞检测
- (5) 光影优化：Blinn-Phong 光照、阴影映射、PCF、Gamma 校正

12. 心得体会

12.1 工期

(1) 2~3 周：学习需要用到的技术。

学习资料：LearnOpenGL 、 GAMES 101 、 GAMES 103 、 相关论文、知乎、b 站。

(2) 1~2 周：代码实现。

① 感谢 gty 同学给我讲解布料仿真中涉及的结构力学。

② 感谢 hzq 同学帮我找到布料仿真中的一个大 bug 。

12.2 OpenGL 的调试方法

(1) 为 shader 的读取、编译、链接设置异常处理，输出对应的日志信息。若程序能运行，但有物体未显示出来，可能是其对应的 shader 读取或编译失败。

(2) 若物体不能正常显示，一般是 vshader 的问题，可检查设置的 uniform 变量的名称是否对应。

(3) 若物体位置能正确显示，但纹理或光照等不正确，一般是 fshader 的问题，可检查变量名是否写错。

(4) 反射光只能在一定范围内看见，若光源和材质的参数设置正确但未见物体的反射，可能是摄像机太高了(高于光源)。

(5) 注意函数的参数向量的方向，如光源指向片段还是片段指向光源。

(6) 将物体加入场景中、对物体做变换时，要考虑是在何坐标系下进行。

(7) 若将顶点坐标、法向量、颜色、纹理坐标等集合在一个 float[] 中，则修改后要更新 glVertexAttribPointer() 中的步长、偏移量等。若程序能过编译但运行崩溃，考虑为此原因。

(8) 若一个物体涉及到多个顶点，如一个弹簧有两个质点，计算空间时要乘上顶点个数。

(9) 若有多个 shader，则设置每个 shader 的 uniform 变量时要注意使用该 shader 的着色器程序对象。

(10) 注意 model 矩阵中平移、旋转、缩放顺序。

12.3 图形学的学习心得

(1) 学图形学就像学 ACM，理论简单实现难，轻理论重实践。

(2) 从学习图形学的角度，最好能看懂所有理论和实现，如深度缓冲等的计算。从编程的角度，无需看懂所有理论和实现，只需将代码贴到正确的位置和修改参数。

(3) GAMES 101 讲得很好，但是课堂上纯理论，几乎不讲实现，不适合新手。LearnOpenGL 理论和实现结合，顺序合理，适合新手入门图形学。SZU 的图形学理论课顺序安排合理，实验课注重实现，更重要地，周老师讲得好。

(4) 多文件，OOP，多封装繁琐的操作，如 shader 类、camera 类、绘制正方体、绘制球等。

(5) 手推几何关系时不能将图画得太特殊。

(6) 要画出好看的场景，除了需要过硬的理论和实现，还需要良好的审美。认知决定上限。

(7) 仿真写起来太恶心了，物理又看不懂，代码又调不对。

(8) LearnOpenGL 的中文翻译有挺多错的，如果发现自己有地方读不懂，可以怀疑是翻译错了。建议对照英文版阅读。

(9) 不迷信权威，如 LearnOpenGL 中关于有透明物体的场景的渲染顺序的介绍有点问题。我认为的顺序: ① 不透明物体、全透明物体; ② 天空盒; ③ 按顺序的半透明物体.

(10) 有人说 OpenGL 很难用，哪里难了，这么多年一直是这个难度。当然还是 Blender 好用多了。

参考资料

- [1] Baraff and Witkin. 1998. Large Step in Cloth Simulation. SIGGRAPH.
- [2] [游戏物理探秘]理解 Large Steps in Cloth Simulation
(<https://zhuanlan.zhihu.com/p/449823510>)
- [3] LearnOpenGL CN (<https://learnopengl-cn.github.io/>)
- [4] GAMES101-现代计算机图形学入门-闫令琪
(<https://www.bilibili.com/video/BV1X7411F744/>)
- [4] Lecture 05 Physics-based Cloth Simulation. GAMES103-基于物理的计算机动画入门
(<https://www.bilibili.com/video/BV12Q4y1S73g?p=5>)
- [5] GAMES103 笔记 Lecture5 基于物理的布料模拟 (Physics-based Cloth Simulation)
(<https://zhuanlan.zhihu.com/p/441088912>)
- [6] 布料仿真-学术界的 30 年 (<https://zhuanlan.zhihu.com/p/560239986>)